CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

Project – Fundamental Topics

A MOBILE PERSONAL FINANCE MANAGEMENT APPLICATION SUPPORTED BY A LARGE LANGUAGE MODEL — PERFIN

Major:	Software Engineering
Cohort:	49
Course class:	CT239H M01
Advisor:	Dr. Phan Phuong Lan
Student:	Nguyen Thanh Trong
Student ID:	B2305615

Semester 3, academic year 2025–2026

Can Tho, 2026

ABSTRACT

Context: Manual entry makes personal-finance histories incomplete or inconsistent, while figures generated by an LLM cannot be written directly to a ledger. **Objective:** This project builds PERFIN, a mobile prototype that accepts natural transaction input while preserving verifiable data and analysis. **Method:** Text, receipt images and speech are converted into typed drafts and written to PostgreSQL only after confirmation. Deterministic functions compute balances, trends, anomalies, cash flow, budgets and goals; the LLM only extracts intent and narrates facts. **Results:** The backend passes 44/44 test files. On 5,265 transactions, the parser obtains 26.12% accuracy and macro-F1 0.1559 using joint type/category labels. In the 63-sentence ablation, full-set accuracy is 22.22% for the parser and 39.68% for Gemini; Gemini returns a category for only 42/63 sentences, and macro-F1 0.6068 is conditional on those 42 answers. Conditional on a correct parser transaction type and after excluding *Other*, group-split correction evaluation (4,756 eligible samples; 76 wrong-type rows excluded) has zero seed/evaluation overlap and raises accuracy from 34.46% to 61.22% (740 helped, 41 harmed, net $+26.76$ points; coverage 891/2,612). **Significance:** The architecture uses an LLM without replacing the data-and-algorithm core, while reporting coverage and harmful effects rather than conditional scores alone. OCR/STT accuracy and production readiness have not been established.

Keywords: personal finance management, large language model, data analysis, PostgreSQL, verifiable systems.

Contents

List of Tables

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BOM	Byte Order Mark
CER	Character Error Rate
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DB	Database
ERD	Entity–Relationship Diagram
F1	Harmonic mean of precision and recall
FK	Foreign Key
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
IQR	Interquartile Range
JSON	JavaScript Object Notation
KV	Key–Value
LLM	Large Language Model
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
OLS	Ordinary Least Squares
PDF	Portable Document Format
PERFIN	Personal Finance — the name of the prototype presented in this report
PK	Primary Key
REST	Representational State Transfer
SQL	Structured Query Language
STT	Speech-to-Text
SUS	System Usability Scale
TC	Test Case — a test-case identifier at the specification level
TTL	Time To Live
UAT	User Acceptance Testing

Abbreviation	Meaning
UI	User Interface
UML	Unified Modeling Language
VND	Vietnamese Dong
WER	Word Error Rate

STATUS AND RESULT CONVENTIONS

This report clearly distinguishes implemented components, measured results and behavior that exists only at the design level. This convention prevents expectations or smoke tests from being presented as completed quality evidence.

Table 1: Convention for levels of information confidence in the report

Label	Meaning	Usage
Implemented	A corresponding component exists in source code, migrations or tests.	Proves existence; does not automatically prove quality.
Measured	A command, timestamp and reproducible run result exist.	May be used to report figures in the results section.
Target	A desired threshold used as an acceptance criterion.	Must not be presented as an achieved result.
Design target	Expected behavior after in-scope fixes are completed.	Must be tested before being converted to “measured”.
Out of scope	Not a goal of the current project.	Mentioned only under limitations or future work.

CHAPTER 1: INTRODUCTION

1.1. PROBLEM STATEMENT

Personal finance management requires regular recording, consistent classification and an understanding of income–expense history. Financial literacy is directly associated with long-term planning and decision-making [1]. Conventional forms, however, require many steps, so omissions or category errors propagate into balances, budgets and reports.

Natural input such as “ăn phở sáng nay 45k bằng Momo” reduces effort but creates new risks. A system must understand Vietnamese currency notation, relative dates, multiple transactions in one sentence, approximate wallet names and classification context. Receipt images and speech introduce additional OCR/STT error. Writing an inferred result directly to the database can therefore corrupt all downstream analysis.

PERFIN follows one principle: PostgreSQL and deterministic functions are the source of truth; LLM, OCR and STT services only produce intermediate data for review. The project addresses three research questions:

1. How can text, image and speech be transformed into structured transactions without compromising data correctness?
2. Which deterministic algorithms are suitable for explainable insights from personal-finance history?
3. Where should an LLM participate to improve natural interaction without taking over computation or modifying data autonomously?

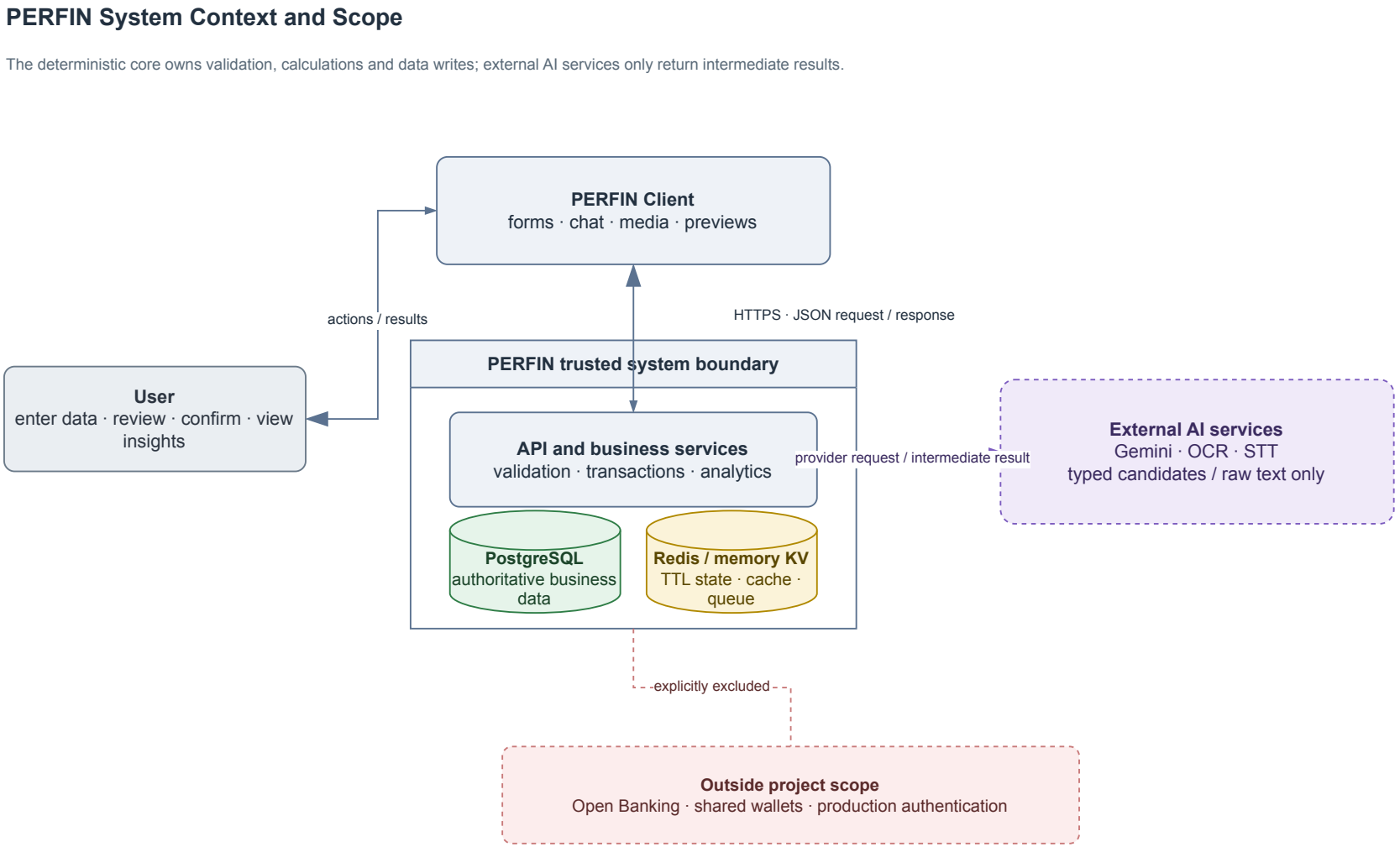

Figure 1: PERFIN system context and scope; LLM, OCR and STT services remain outside the trusted data boundary.

Figure 1 limits the project to acquiring, organizing and analysing personal-finance data. Bank connectivity, shared wallets, production authentication and high-availability infrastructure are out of scope.

1.2. OBJECTIVES

The overall objective is to complete, within 13 weeks, a personal-finance prototype in which structured data and deterministic algorithms form the foundation and the LLM is a controlled language-understanding and narration layer.

Table 2: Specific objectives, acceptance criteria and deadlines

Code	Specific objective	Evaluation criterion	Deadline
O1	Build a financial data model with keys, constraints and atomic updates.	Migrations, ERD and rollback tests; no partial write in critical flows.	Week 5
O2	Build text, image and speech extraction with clarification, preview and confirmation.	No database write when fields are missing; retain source and intermediate data.	Week 8
O3	Implement trend, anomaly, runway, recurring, correlation, budget and goal algorithms.	Normal, boundary and invariant cases agree with hand-computed results.	Week 9
O4	Restrict the LLM to intent understanding, tool calling and fact narration.	Tool schema, validation, fallback and confirmation remain separate from write authority.	Week 10
O5	Evaluate the prototype with reproducible tests and experiments.	Report unit/integration/smoke scope, joint metrics, coverage/abstention and correction effects; disclose missing measurements.	Week 13

The criteria in Table 2 are planned acceptance conditions. A result is called "measured" only when its input, environment and artifact are available.

1.3. RESEARCH METHODS

The project combines four methods:

1. **Literature review:** synthesize relational data, text similarity, explainable statistics, OCR, STT and function calling to establish the LLM boundary.

2. **Requirements and systems analysis:** study the input–confirmation–analysis process and model actors, data, constraints and ledger-corruption failure modes.
3. **Prototype design:** develop in short iterations; separate route–service–model responsibilities and implement formulas as independently testable deterministic functions.
4. **Experimentation:** use unit, integration and system smoke tests plus labelled datasets to compare the parser with the LLM, measure correction retrieval and analyse errors. Media smoke tests demonstrate execution only, not accuracy.

1.4. IMPLEMENTATION PLAN

The plan starts on 11 May 2026 and spans 13 weeks. Testing and report writing may overlap, but each week has a verifiable deliverable.

Table 3: Thirteen-week implementation plan

Week	Dates	Main work	Deliverable
1	11–17 May	Review the problem, guideline and related applications.	Research questions and scope.
2	18–24 May	Elicit requirements and normalize the SRS.	FR/NFR list and overall use case.
3	25–31 May	Design architecture and the LLM boundary.	Architecture diagram and module contracts.
4	01–07 Jun	Design the domain and relational schema.	Class diagram, ERD and migration draft.
5	08–14 Jun	Implement ledger, constraints and transactions.	O1 and core-data tests.
6	15–21 Jun	Implement parser, normalization and fuzzy matching.	Text pipeline and quality gate.
7	22–28 Jun	Integrate LLM, clarification and pending state.	Tool schema and preview/confirm flow.
8	29 Jun–05 Jul	Integrate OCR/STT through adapters.	Media pipeline and provider-error handling.
9	06–12 Jul	Implement analytics, budget and goal planning.	O3 and formula unit tests.

Week	Dates	Main work	Deliverable
10	13–19 Jul	Complete narration, feedback and worker logic.	O4, fallback and side-effect controls.
11	20–26 Jul	Run integration tests, import data and conduct experiments.	Logs, result JSON and error analysis.
12	27 Jul–02 Aug	Restructure the report and normalize diagrams.	Guideline-compliant LaTeX v2.
13	03–09 Aug	Review, compile, repair references and prepare the demo.	Submission PDF, source and user guide.

1.5. PROJECT SCOPE

The implementation includes income and expense transactions, wallet transfers, categories, budgets, recurring bills, goals, data analysis and prototype-level text/image/speech input. Evaluation focuses on the data model, transformation correctness and algorithms. The mobile interface captures input and demonstrates the workflow.

Open Banking, foundation-model training, shared wallets, investment advice, production authentication and authorization, high availability and uptime commitments are excluded. The prototype uses `default_user`; user foreign keys indicate an extension path, not evidence of secure multi-user operation.

1.6. CONTRIBUTIONS

The principal contributions are a constrained data model with atomic updates; a multimodal human-in-the-loop pipeline; independently testable analytics; an LLM boundary limited to extraction and narration; and an evaluation protocol that separates algorithm quality, provider execution and system readiness.

1.7. REPORT STRUCTURE

Chapter 1 presents the problem, objectives, methods and plan. Chapter 2 summarizes theory, compares alternatives and explains technology choices. Chapter 3 presents the SRS, design, detailed algorithms and testing. Chapter 4 reviews the objectives, limitations and future work.

CHAPTER 2: THEORETICAL BACKGROUND

This chapter summarizes the concepts, models and technologies used by the project. Project-specific formulas and thresholds are moved to the detailed design in Chapter 3.

2.1. FINANCIAL DATA AND PREPROCESSING

2.1.1. Relational Ledger and Data Integrity

A financial transaction minimally contains an amount, type, date, description, category and wallet. Income increases a balance, expense decreases it, and a wallet transfer creates two opposite changes that must not be counted as income or expense. Multi-record changes belong in one database transaction so they either complete or roll back together.

A relational database provides foreign keys, decimal types, domain constraints and aggregate queries. Durable business data must remain separate from short-lived clarification, preview, cache and queue state. Analyses defined on a fixed calendar axis zero-fill inactive periods: trends use completed months, correlations use completed ISO weeks, and runway/anomaly preserve every calendar day. Partial current months/weeks are excluded from comparisons; dropping zero periods distorts slopes, burn rate and correlation.

2.1.2. Normalization and Text Similarity

Input is normalized for case, punctuation, whitespace, currency units and relative dates before schema mapping. For strings a, b, Levenshtein similarity [2] is

$$s_{edit} = 1 - \frac{D_{lev}(a, b)}{\max(|a|, |b|)}. \tag{2.1}$$

For token sets T_a, T_b, the Dice coefficient [3] is

$$s_{dice} = \frac{2|T_a \cap T_b|}{|T_a| + |T_b|}. \tag{2.2}$$

Levenshtein distance captures nearby typing errors, whereas Dice is useful when phrase order or components differ. PERFIN combines exact matches, aliases, both scores and a safety margin between the two best candidates. The chosen thresholds are detailed in Section 3.2.3.2.

2.2. EXPLAINABLE ANALYTICAL TECHNIQUES

2.2.1. Trend, Anomaly and Correlation

Ordinary least squares (OLS) [4] models a time series as $\hat{y}_i = a + bx_i$. Its slope is

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \tag{2.3}$$

The sign of b gives direction and R^2 gives linear fit. OLS is explainable and testable but does not model seasonality or future events. A z-score compares an observation with the mean in standard-deviation units, while the interquartile range is more robust to extremes [5]. PERFIN uses both as review signals, not as fraud conclusions.

Pearson's coefficient [6] measures linear co-movement:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{2.4}$$

Correlation is interpreted only within the observed window and does not imply causation. The coefficient is undefined with fewer than three paired observations or when either series has zero variance; that case must be represented as a null value rather than being reported as $r = 0$.

2.2.2. Runway, Recurring Items, Budgets and Goals

Runway extrapolates how long liquid balance in the reporting currency can support the recent spending rate; it is not total wealth. Recurring-item discovery groups transactions by description, amount and date interval. Budget forecasting uses spending pace over elapsed days, while goal planning uses the remaining amount, deadline, contribution and debt simulation. These techniques prioritize transparent assumptions and boundary handling. PERFIN's formulas and windows appear in Section 3.2.3.3.

Table 4: Selected analytical techniques

Technique	Role	Reason for selection
OLS	Trend and short-horizon forecast.	Few parameters; slope and fit are explainable.
Z-score + IQR	Unusual amounts or days.	Combines distribution sensitivity with quantile robustness.
Runway	Cash-depletion warning.	Direct calculation with explicit assumptions.
Rule-based grouping	Recurring-spending discovery.	Suits small data and requires no training.
Pearson	Co-moving category pairs.	Standardized coefficient with clear interpretation.
Budget/Goal planner	Limits, forecasts and progress.	Hand-checkable results and what-if simulation.

2.3. OCR, STT AND LANGUAGE MODELS

2.3.1. OCR and Speech-to-Text

OCR generally includes image preprocessing, text-region detection, recognition and post-processing; Tesseract is a representative architecture [7]. STT converts audio to a transcript, and Whisper illustrates recognition learned from large-scale data [8]. In PERFIN, both services only return raw text and provider metadata. The result follows the same normalization, preview and confirmation pipeline as manual input.

Quality requires ground truth. CER/WER measure text error, while transaction-field accuracy measures the effect on amount, date, category and line items. A smoke test on a few files establishes provider execution, not accuracy.

2.3.2. Local Parser and LLM

Transformers use attention to model token relationships and underpin modern LLMs [9]. Multimodal models such as Gemini extend this ability to text and media [10]. Structured output constrains shape but cannot prove that an amount is correct, a category belongs to the user or an operation is authorized.

Table 5: Comparison of the local parser and an LLM

Criterion	Local parser	LLM via function calling
Language coverage	Strong on known patterns; weak on free-form input.	Better at variations and context.
Reproducibility	High, offline and no API cost.	Model/provider dependent and slower.
Control	Rules and failures are traceable.	Requires locked schema, validation and fallback.
PERFIN role	Baseline, fallback and quality gate.	Primary extraction/routing when configured.
Business authority	Returns a typed draft; never writes directly.	Proposes a tool and arguments; never writes directly.

PERFIN uses provider-schema function calling [11]. The LLM proposes a tool and arguments; the backend validates, builds a preview and waits for confirmation. For insight narration, the LLM only receives precomputed facts. This preserves interaction benefits without making the model a source of truth.

2.4. TECHNOLOGIES AND RATIONALE

Table 6: Technologies, roles and alternatives

Technology	Reason for selection	Limitation or alternative
React Native + Expo	One codebase for forms, chat, image, audio and mobile preview.	Separate native apps are only needed for device-specific requirements.
Node.js + Express	Fits I/O-oriented APIs and shares JavaScript with business functions and tests.	A modular monolith is sufficient; microservices add operations cost.
PostgreSQL [12]	Foreign keys, `DECIMAL`, transactions and aggregates fit a ledger.	File/NoSQL designs provide weaker constraints and rollback.
Redis [13]	TTL for pending state, clarification, cache and queue data.	Never the system of record; memory fallback is development-only.
BullMQ [14]	Scheduling, retry and job IDs for idempotent work.	Basic cron lacks equivalent queue and retry controls.

Technology	Reason for selection	Limitation or alternative
Gemini + local parser	Combines language coverage with a testable fallback.	Provider adapters prevent business-logic lock-in.
Local or cloud OCR/STT	PaddleOCR/PhoWhisper favour privacy; cloud providers remain replaceable.	Every raw output still follows validation and preview.

2.5. RELATED APPLICATIONS AND SELECTED DIRECTION

Money Lover [15] and MISA MoneyKeeper [16] represent form- and dashboard-oriented finance applications. They are strong at structured records but require many entry steps. Chatbots reduce entry work yet risk untraceable numbers; specialist analytics are often difficult for general users.

PERFIN does not attempt commercial feature completeness. It targets the intersection of a relational ledger, deterministic algorithms, human confirmation and a bounded language layer. This direction fits the project's data-and-algorithm focus and allows each part to be evaluated independently.

Table 7: Research gaps and PERFIN responses

Gap	Risk	Selected response
Natural input may be miswritten	Dirty data propagates to reports	Typed draft, validation, preview and confirmation.
Media provides raw text only	Wrong amount, date or category	Ground truth, field accuracy and confirmation.
Chatbot numbers lack provenance	Hallucination and weak auditability	SQL/analytics creates facts; narrator only explains.
Classification does not adapt	Repeated errors	Thresholded, controlled correction retrieval.
Retry duplicates effects	Corrupt data or duplicate alerts	Job IDs, fingerprints and idempotent operations.

CHAPTER 3: APPLICATION RESULTS

3.1. SOFTWARE REQUIREMENTS SPECIFICATION

3.1.1. Overall Description

PERFIN is a mobile prototype for entering, normalizing, managing and analysing personal-finance data. Chat and media complement forms as input channels. The backend performs every financial calculation, constraint check and data change; LLM, OCR and STT services have no direct PostgreSQL access and are not authoritative numeric sources. FR/NFR identifiers and test traceability follow the requirements-engineering guidance of ISO/IEC/IEEE 29148:2018 [17].

Table 8: Product context and constraints

Item	Description
User	A general user records and interprets personal-finance data without requiring statistical expertise.
Environment	React Native/Expo on mobile or web, Express API and PostgreSQL; Redis and a worker are configuration-dependent.
System of record	PostgreSQL stores business data; Redis stores only TTL-bound pending, clarification, cache and queue state.
Constraints	The demo uses `default_user`, has no production authentication/authorization, defaults to VND and has no foreign-exchange conversion.
Dependencies	LLM/OCR/STT providers are replaceable; failure must produce a real error or fallback, never fabricated data.
Assumption	The user reviews previews; guidance does not replace professional financial advice.

Table 9: Actors and concerns

Actor	Role	Concern or limitation
User	Enters, edits, confirms, manages and views reports.	Language/media-derived data must be reviewed before storage.
External AI service	Returns a typed draft, OCR text or transcript.	Cannot execute business operations or write the database.
Background worker	Runs reminders, analysis and file cleanup.	Jobs require user scope, retry and duplicate-effect protection.
Developer administrator	Configures environment, migrations and providers.	Not a business actor of the application.

Shared rules are: business amounts must be finite and valid; transaction dates cannot be in the future; wallets and categories belong to the current profile; negative wallet balances are allowed; chat/media operations require preview and confirmation; multi-record updates are atomic; reports exclude soft-deleted transactions by default; and insufficient data yields a warning or `null`, never an invented value.

3.1.2. Functional Requirements

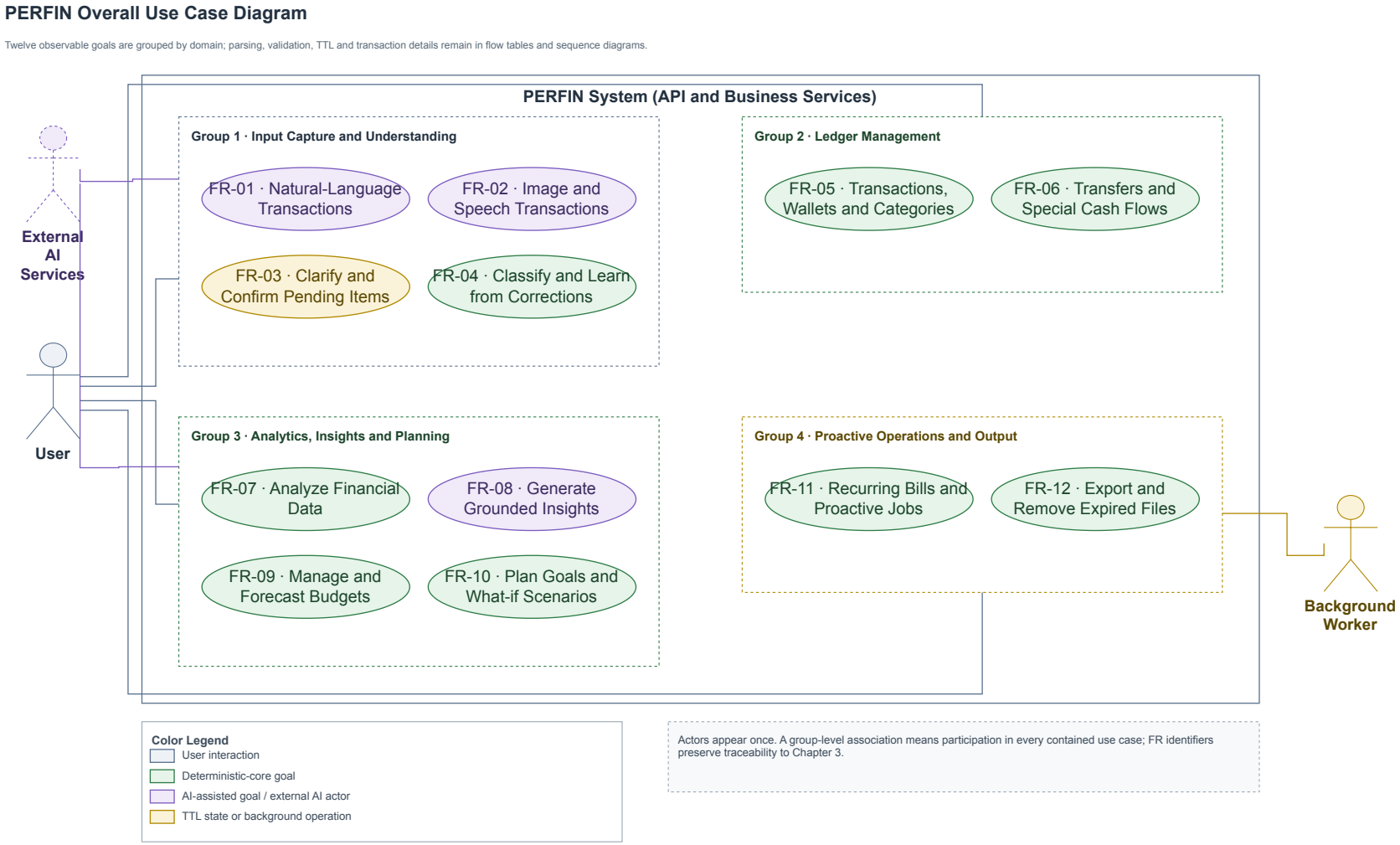

Figure 2: Overall PERFIN use case diagram. Feature details are specified with flow tables instead of twelve separate diagrams.

Figure 2 contains observable actor goals only. Parser, validation, transaction, TTL and retry behaviour belongs in the flow tables or the sequence/flow diagrams in Section 3.2.3.

Table 10: FR-01 — Enter Transactions from Text

Item	**Description**
Feature name	Enter transactions from natural-language text.
ID	FR-01.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user needs fast entry; the AI Orchestrator returns a typed draft; Transaction Service protects the ledger.
Summary	Convert one sentence into one or more structured transactions and create a preview before writing.
Relationships	Includes FR-03 and FR-04; uses FR-05 after confirmation.
Normal flow	**Step 1:** User submits text. **Step 2:** Extract and normalize. **Step 3:** Validate schema, category and wallet. **Sub 1:** Build preview. **Step 4:** User confirms. **Step 5:** Commit atomically and return the result.
Subflows	**Sub 1 — Preview:** (1) Store a TTL-bound draft; (2) show amount, type, date, category, wallet and warnings.
Alternate flows	**Step 2:** Provider failure uses the local parser or returns an error. **Step 3:** Missing/ambiguous fields invoke FR-03. **Step 4:** Edit or cancel causes no write.

Table 11: FR-02 — Enter Transactions from Image or Speech

Item	**Description**
Feature name	Enter a transaction from a receipt image or speech.
ID	FR-02.
User	User; External AI service.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs visible raw text; adapters report the provider and failures; the backend rejects invalid files.

Item	Description
Summary	Convert image/audio into reviewable text, then reuse FR-01.
Relationships	Includes FR-01 and FR-03.
Normal flow	**Step 1:** Select a valid file. **Step 2:** Backend invokes OCR or STT. **Step 3:** Display raw text and provider. **Sub 1:** User edits/confirms text. **Step 4:** Continue with FR-01.
Subflows	**Sub 1 — Media review:** (1) edit a voice transcript; (2) for a receipt, select total or individual items when present.
Alternate flows	Invalid type or size above 10 MB is rejected. A provider that returns no text produces an error and no mock or pending draft.

Table 12: FR-03 — Clarify and Confirm Pending Transactions

Item	Description
Feature name	Clarification, preview and pending-transaction confirmation.
ID	FR-03.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user can edit/cancel; the KV store applies TTL; a preview can be consumed once.
Summary	Collect missing fields across turns and prevent inferred data from being written.
Relationships	Used by FR-01, FR-02, FR-09, FR-10 and FR-12.
Normal flow	Store intent and missing fields; ask for the next field; merge and revalidate; build/edit a preview; atomically claim pending state; call the business service.
Subflows	**Preview:** attach a `pending_id` with five-minute TTL and update only with a valid payload.
Alternate flows	Continue asking while fields are missing. Expired, incorrect or claimed IDs cannot write; cancellation clears state.

Table 13: FR-04 — Classify and Learn from Corrections

Item	Description
Feature name	Category classification and correction retrieval.
ID	FR-04.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The matcher must be explainable; the user corrects labels; feedback failure must not corrupt a committed transaction.
Summary	Select a category by exact/alias/fuzzy/LLM methods and reuse confirmed corrections.
Relationships	Extends FR-01 and may be triggered by an FR-05 category edit.
Normal flow	Normalize description; load same-profile categories/corrections; rank candidates; apply a strong correction; return category, confidence and match kind; store feedback after confirmation.
Subflows	Select at most five similar corrections and use them as context or a controlled override.
Alternate flows	Low score, small margin or conflicting corrections return Other/request a selection. Feedback-write failure is logged only.

Table 14: FR-05 — Manage Financial Data

Item	Description
Feature name	Manage transactions, wallets and categories.
ID	FR-05.
User	User.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	The user needs CRUD and restore; services maintain balance and history consistency.
Summary	Create, view, edit, soft-delete and restore transactions and manage profile-scoped categories/wallets.
Relationships	Used by FR-01/FR-02 and supplies FR-07–FR-12.

Item	Description
Normal flow	Select operation; validate payload and ownership; compute balance delta; update transaction and wallet atomically; return record and new balance.
Subflows	Write the transaction, update the wallet, commit, then invalidate cache best-effort.
Alternate flows	Out-of-scope IDs or invalid payloads are rejected. A negative balance remains valid. Any pre-commit failure rolls back.

Table 15: FR-06 — Transfer Funds and Record Special Cash Flows

Item	Description
Feature name	Wallet transfer and investment gain/loss.
ID	FR-06.
User	User.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	One user action must update both wallets and history consistently.
Summary	Record internal transfers without changing total assets and classify investment cash flow correctly.
Relationships	Uses FR-05 wallet and transaction entities.
Normal flow	Enter source, target, amount and date; validate distinct wallets; lock in stable order; debit, credit and write history; commit.
Subflows	In one transaction, subtract source, add target and write `wallet_transfers`; net change equals zero.
Alternate flows	Same, missing or out-of-scope wallets, invalid amount/date, or different wallet currencies roll back. A future negative source balance alone is not rejected.

Table 16: FR-07 — Analyse Financial Data

Item	Description
Feature name	Trend, anomaly, runway, recurring and correlation analysis.

Item	Description
ID	FR-07.
User	User; Background worker.
Necessity	Mandatory.
Classification	Complex.
Participants and concerns	Analytics needs clean data; users need period, sample size and method.
Summary	Compute deterministic facts over non-deleted data; trend/correlation use completed periods and fixed calendar axes are zero-filled.
Relationships	Reads FR-05 and supplies FR-08 and FR-11.
Normal flow	Select analysis and period; query profile data; normalize the calendar axis and check sample count; run the algorithm; return facts with window, method, parameter and warning metadata.
Subflows	Run OLS, z-score/IQR, runway, recurring or Pearson through one result contract.
Alternate flows	Sparse data or zero variance returns a degraded result or `null`; no causal claim or fabricated transaction.

Table 17: FR-08 — Generate Grounded Insights

Item	Description
Feature name	Grounded insight and persona narration.
ID	FR-08.
User	User; External AI service.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs clear explanation; Analytics owns facts; the narrator cannot alter numbers.
Summary	Explain precomputed facts and return them with the narration for traceability.
Relationships	Includes FR-07 and may use a persona when consent is enabled.
Normal flow	Receive a question/report request; generate facts; load a permitted persona; narrate facts; return narration, facts and provider metadata.

Item	Description
Subflows	Lock numbers/units in the prompt and use a deterministic template when the provider fails.
Alternate flows	Sparse facts return a warning. A complete numeric-grounding checker remains a design target and is not presented as measured.

Table 18: FR-09 — Manage and Forecast Budgets

Item	Description
Feature name	Monthly budgets, recommendations and forecasts.
ID	FR-09.
User	User.
Necessity	Mandatory.
Classification	Medium.
Participants and concerns	Users need progress visibility; Budget Engine needs sufficient history and must not apply limits automatically.
Summary	Compute spent/remaining, forecast month end and propose limits from history or a ratio framework.
Relationships	Reads FR-05 and may invoke FR-03 from chat.
Normal flow	Select month; query spending; compute progress and forecast; produce warning/recommendation; confirm before saving a recommendation.
Subflows	Zero-fill months; compute limits; proportionally shrink raw allocations and reconcile rounding when a group exceeds its cap.
Alternate flows	Fewer than three history months produce a warning. A non-positive amount or non-expense category is rejected.

Table 19: FR-10 — Plan Goals and What-if Scenarios

Item	Description
Feature name	Savings, purchase and debt-payoff goal planning.
ID	FR-10.
User	User.
Necessity	Mandatory.
Classification	Complex.

Item	Description
Participants and concerns	The user needs formula-backed progress; Goal Planner must expose infeasible assumptions.
Summary	Calculate contribution, duration, amortization and what-if plans without modifying the original until confirmation.
Relationships	Reads FR-05/FR-07 and may use FR-03.
Normal flow	Enter goal; validate amount/date/rate; compute allocatable cash; run goal-specific formula; return plan and warnings; save after confirmation.
Subflows	What-if reruns the saving/purchase function with an additional contribution; debt payoff uses monthly simulation. The original plan is unchanged and no scenario is persisted.
Alternate flows	Invalid inputs request correction. Payment less than or equal to first-month interest warns of negative amortization; zero payment and a 600-month horizon are reported explicitly. Cancel causes no write.

Table 20: FR-11 — Manage Recurring Bills and Proactive Jobs

Item	Description
Feature name	Recurring bills, payments and proactive messages.
ID	FR-11.
User	User; Background worker.
Necessity	Desirable.
Classification	Complex.
Participants and concerns	The user needs correct due dates; the worker needs retry and idempotency; PostgreSQL must prevent duplicate effects.
Summary	Manage schedules, record one payment per period and generate grounded reminders/alerts.
Relationships	Reads FR-05/FR-07; may call FR-08 for wording only.
Normal flow	Scheduler enqueues a user-scoped job; worker loads due data; computes deterministic facts; persists an event by unique key; advances schedule after success.
Subflows	Payment writes payment history, transaction and next due date atomically.

Item	Description
Alternate flows	Duplicate event/job returns the existing result. Transient failure retries with backoff. Redis failure stops the worker while the interactive API may continue.

Table 21: FR-12 — Export Data and Remove Expired Files

Item	Description
Feature name	Data export, export history and expired-file cleanup.
ID	FR-12.
User	User; Background worker.
Necessity	Optional.
Classification	Medium.
Participants and concerns	The user needs correct data; the exporter needs safe escaping; cleanup cannot remove valid files.
Summary	Generate filtered CSV, store history and delete files after TTL.
Relationships	Reads FR-05, may use FR-03 from chat, and delegates cleanup to the worker.
Normal flow	Select format/period; validate and query profile data; create a safe file; write `export_history`; return a download link.
Subflows	Escape formula-like cells, write UTF-8 BOM and row count, then assign TTL.
Alternate flows	Empty data returns a header-only file. The path once labelled PDF only creates HTML and is not claimed as a true PDF export.

3.1.3. Non-functional Requirements

Table 22: Non-functional requirements and acceptance methods

ID	Group	Testable requirement	Measurement
NFR-01	Recovery	Provider/Redis failure fabricates no data; restore demo service and backup data within three hours after an incident.	Fault injection, restart and restore on a test database.

ID	Group	Testable requirement	Measurement
NFR-02	Integrity	No partial create, update, delete, restore, transfer or payment; credentials are never transmitted or logged in clear text.	Rollback tests and configuration/log scanning; TLS for network deployment.
NFR-03	UI	Vietnamese blue–white interface, consistent navigation and no overflow at 390×844.	UI smoke on Dashboard, Transaction and Chat.
NFR-04	Timing	Acknowledge within three seconds; target text p95 ≤ 3 s and media p95 ≤ 8 s in a disclosed environment.	At least 30 runs per flow with provider, hardware, cache and error rate.
NFR-05	Correctness	Algorithms match expected normal/boundary cases; negative balances are valid; narrator invents no number.	Unit tests, invariants and facts–response checker.
NFR-06	Privacy	Remove temporary media; use traits only with consent; ID queries include user scope.	Temporary-file, consent and cross-scope tests.
NFR-07	Worker consistency	Retrying an event/job creates no duplicate payment, message or export.	Repeat fingerprints and count effects.
NFR-08	Traceability	Analysis includes schema version, timestamp, component value and metadata for unit, window, sample count, method, parameters, warnings and degradation status.	Contract tests and FR–test–artifact matrix.

Recovery, latency, OCR/STT accuracy and complete numeric grounding remain acceptance targets; unmeasured targets are not presented as achieved results.

3.2. SOFTWARE DESIGN

3.2.1. Architecture

PERFIN is a modular monolith: domain services are separated by responsibility while the API remains one process; a separate worker handles background jobs. This matches project scale, reduces operating cost and preserves independent module tests.

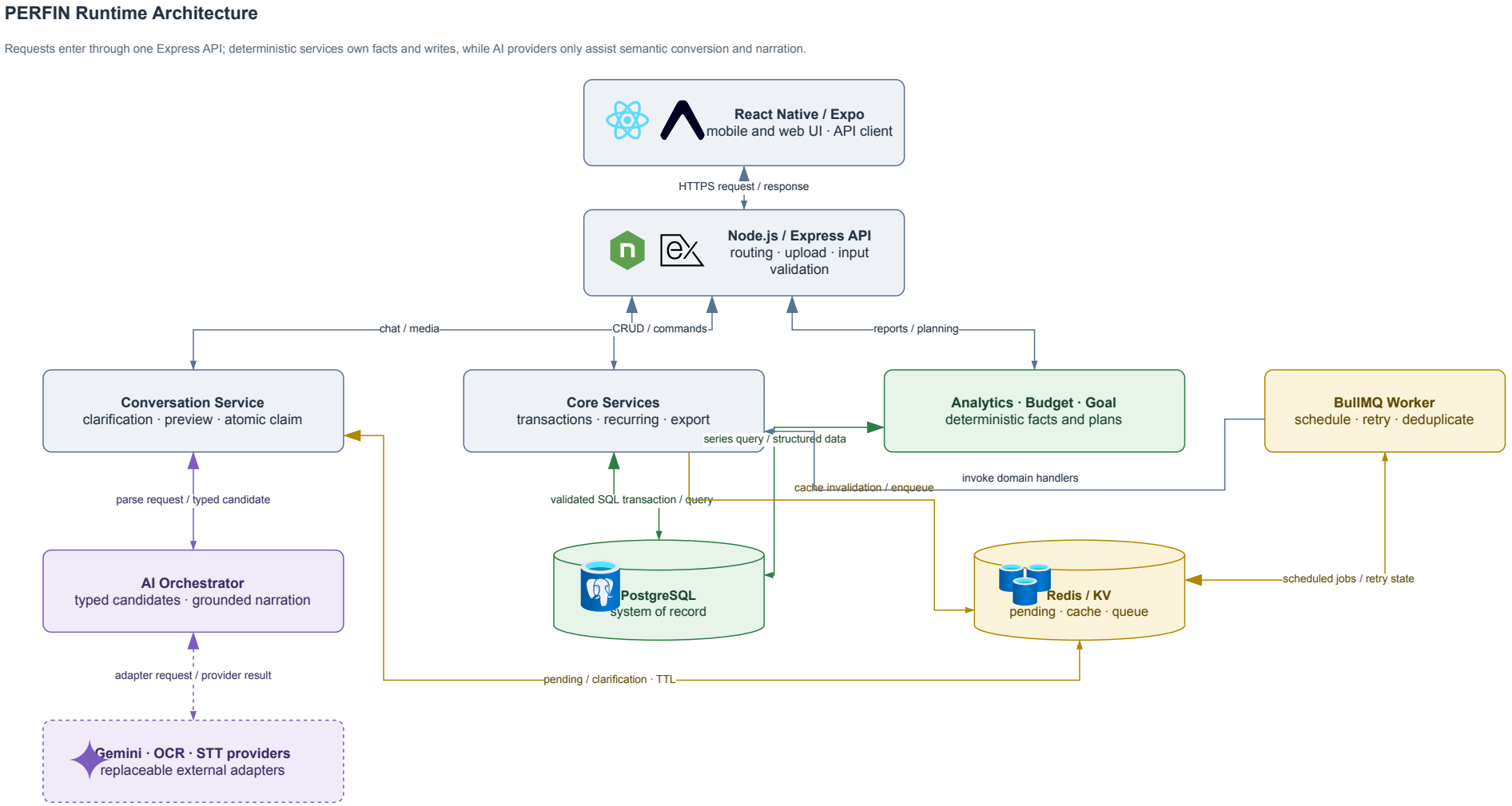

Figure 3: PERFIN runtime architecture; the deterministic core creates and stores facts while the AI Orchestrator coordinates semantics.

Table 23: Architectural components and responsibilities

Component	Responsible for	Not responsible for
Mobile App	Capture input, show previews/facts and confirm.	Database access or AI secrets.
Express Routes	HTTP contracts, upload and input validation.	Analytical formulas.
Core Services	Business rules, transactions and idempotency.	LLM-generated numbers.
Analytics, Budget, Goal	Deterministic facts and plans.	Arbitrary advice.
AI Orchestrator	Tool routing, extraction, narration and fallback.	Direct database writes.
PostgreSQL	System of record, constraints and aggregates.	Temporary conversation state.
Redis/BullMQ	TTL state, cache, queue and retry.	Financial source of truth.

Microservices were considered but not selected because prototype scale does not justify deployment, tracing and distributed-consistency costs. Serverless is also a poor fit for a long-running worker and local media providers. A modular monolith keeps database transactions simple and leaves a future extraction path when real load evidence exists.

3.2.2. Data Design

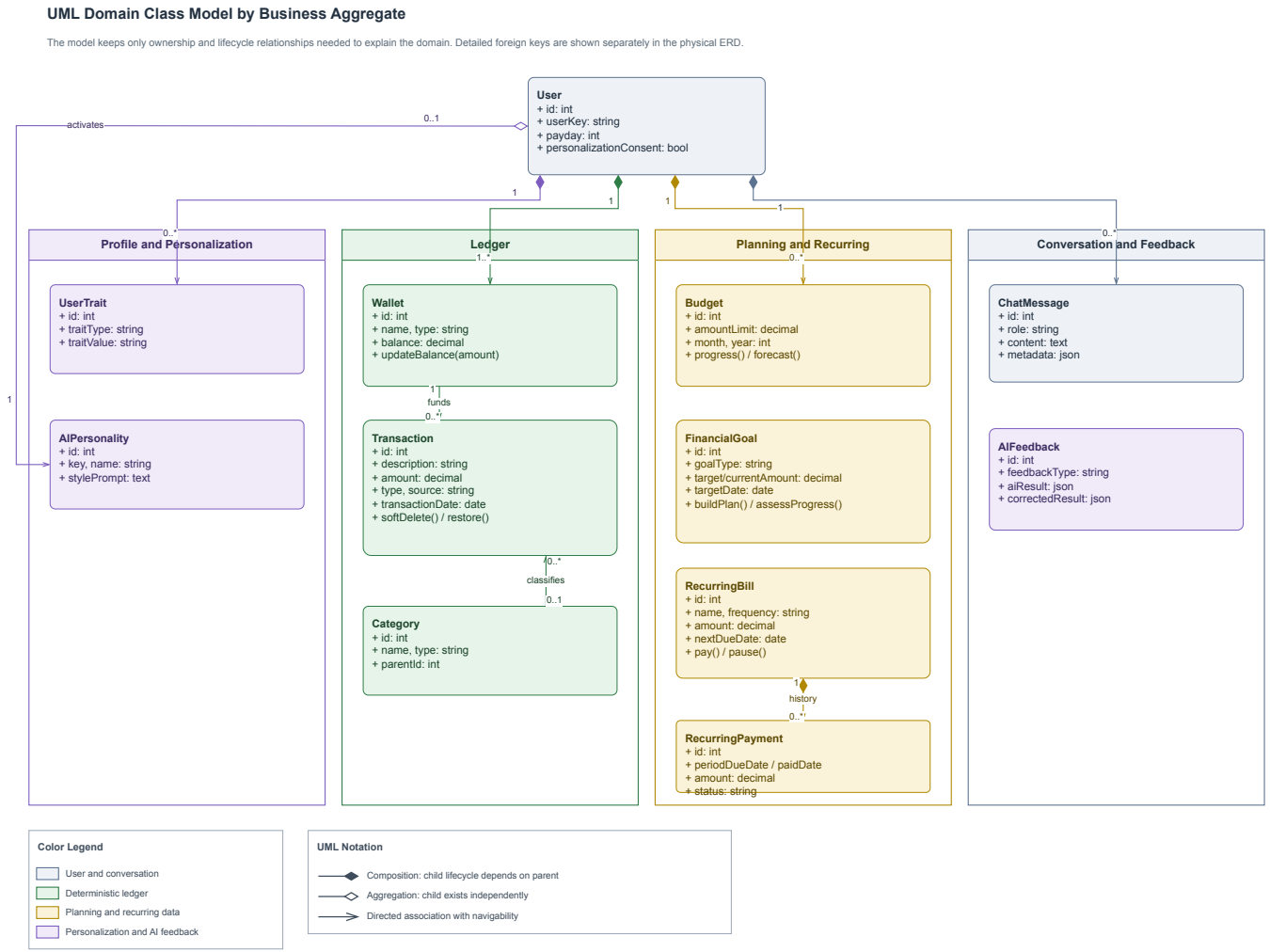

Figure 4: PERFIN UML domain class model separating ledger, planning, recurring and conversation/feedback aggregates.

`User` is the ownership root; `Wallet`, `Transaction` and `Category` form the ledger; `Budget`, `FinancialGoal` and recurring classes form planning; `ChatMessage` and `AIFeedback` preserve traceable interaction. Engines are not entities because they only read data and create facts.

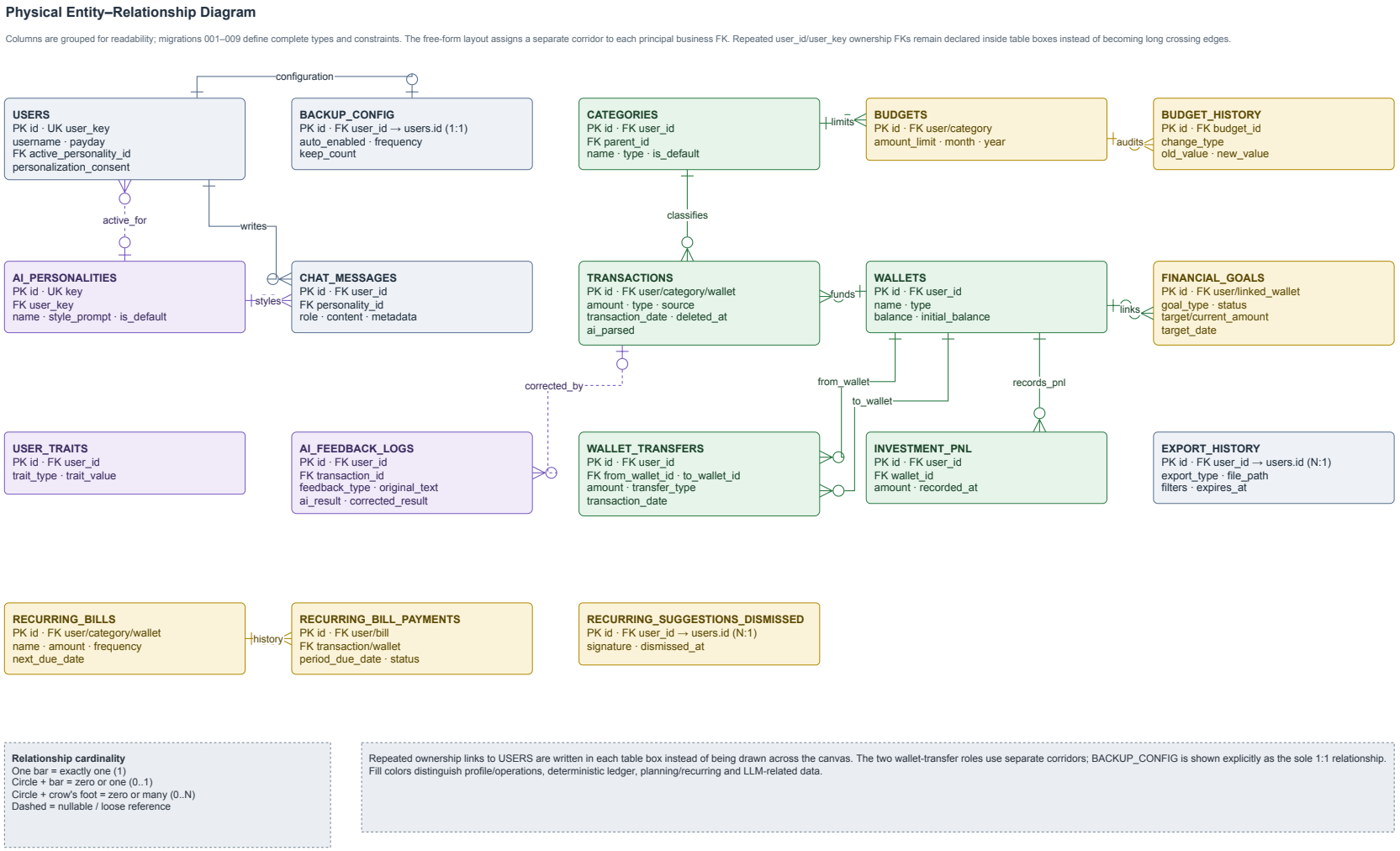

Figure 5: Physical ERD reconciled with runtime migrations; principal business foreign keys use separate orthogonal corridors.

Table 24: Entities in alphabetical order

Entity	Type	Description
ai_feedback_logs	History	Original AI result, correction and context.
ai_personalities	Catalogue	Selectable persona and style prompt.
backup_config	Configuration	Per-user backup policy; worker backup is not proven.
budget_history	History	Audit trail of budget-limit changes.
budgets	Planning	Category and monthly limits.
categories	Catalogue	Income/expense taxonomy and parent relation.
chat_messages	History	Conversation, facts/metadata and event key.
export_history	Operations	Exported files, filters and expiry.
financial_goals	Planning	Goal, progress, deadline and linked wallet.
investment_pnl	Ledger	Realized investment gain/loss.
recurring_bill_payments	Ledger	Payment record for each recurring period.
recurring_bills	Planning	Amount, frequency and next due date.
recurring_suggestions_dismissed	History	Signature of a dismissed recurring suggestion.
transactions	Ledger	Income/expense, input source and soft-delete status.
user_traits	Personalization	Consent-dependent user traits.
users	Data root	Demo profile and data-scope key.
wallet_transfers	Ledger	Transfer between two wallets.
wallets	Ledger	Wallet balance, type and name.

Money uses DECIMAL(15,2); report queries exclude soft-deleted records; deleting a referenced category is blocked or controlled by re-tagging; pending state exists only in Redis; and target-design ID queries include user_id/user_key. Internal transfer requires matching wallet currencies. Runway sums only liquid VND cash,

`bank` and `e_wallet` balances; foreign-currency, savings, investment and credit-card wallets are excluded because the prototype has neither exchange-rate nor liquidity modelling.

3.2.3. Detailed Design

3.2.3.1. LLM Boundary and Input

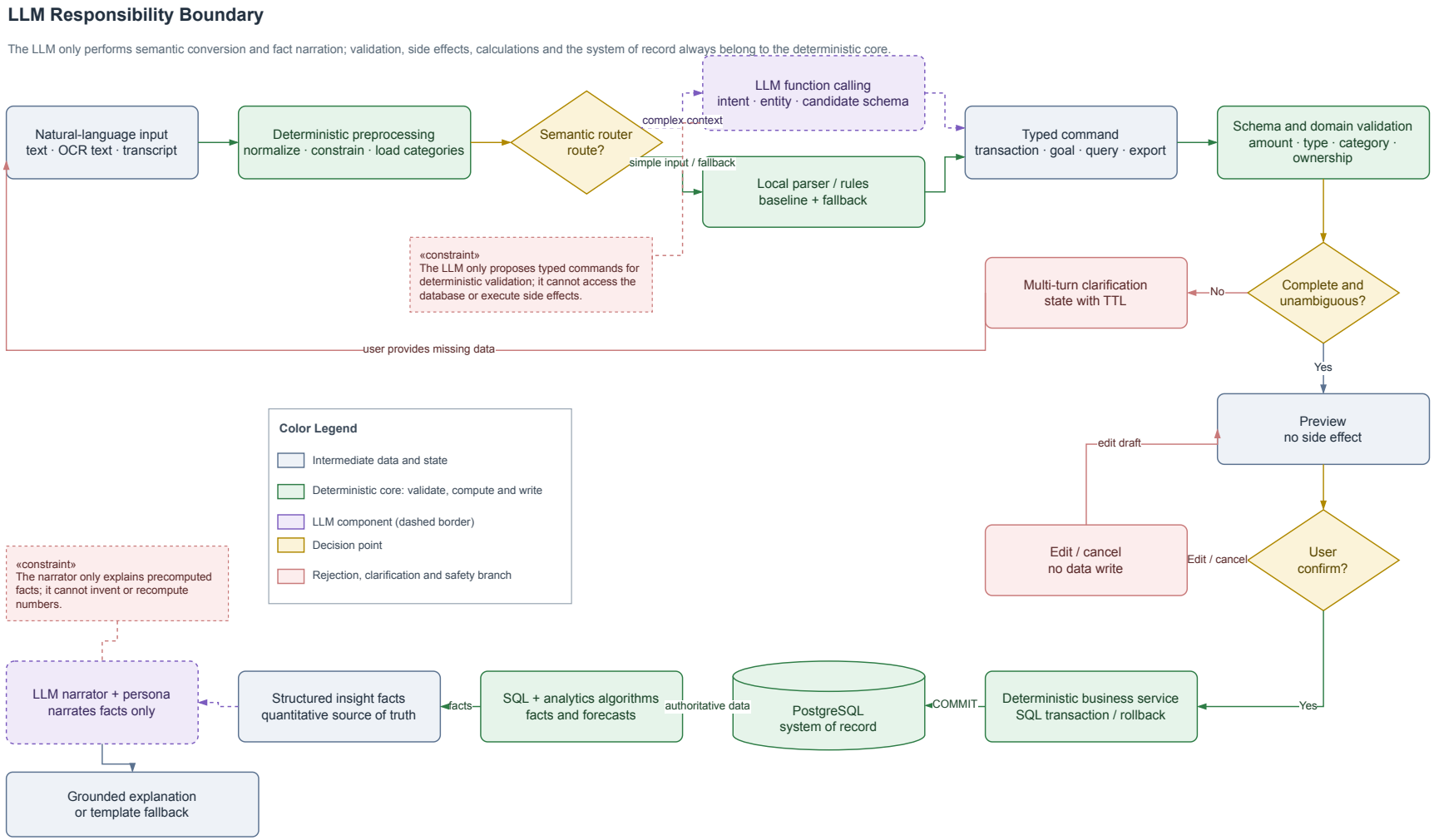

Figure 6: LLM responsibility boundary: typed drafts and narration remain outside validation, calculation and data-write authority.

The LLM only creates a typed command or narrates facts. The backend validates schema and domain rules, builds a preview and waits for confirmation before opening a transaction. For analysis, SQL and the engine create facts before narration. A parser or deterministic template provides a testable fallback.

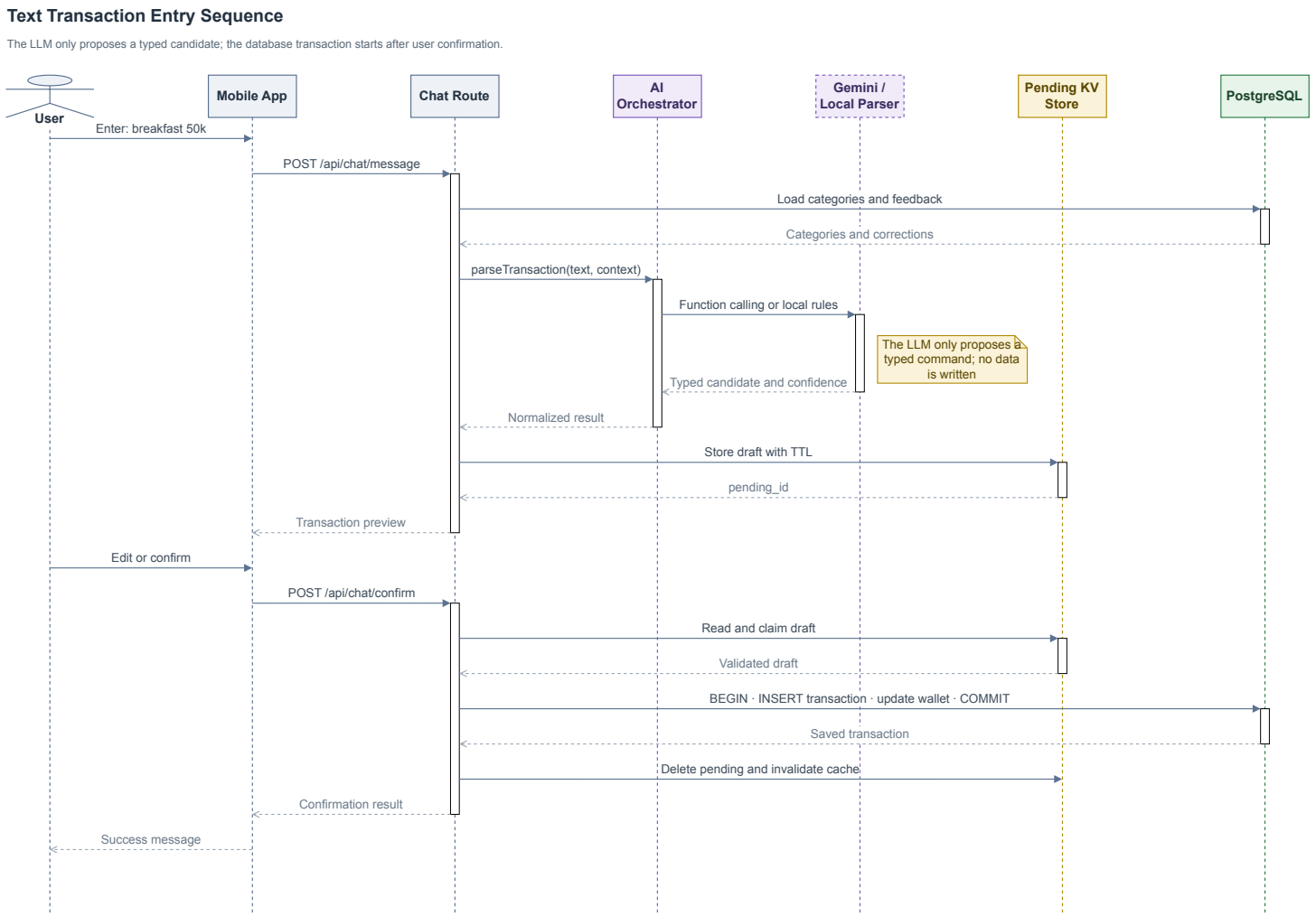

Figure 7: Text-entry sequence; the data-write transaction starts only after confirmation.

Parsing/preview has no side effect; confirm/commit atomically claims pending state. State contains intent, missing fields, candidates and a draft with a five-minute TTL. Concurrent confirmation must not create two transactions from one `pending_id`.

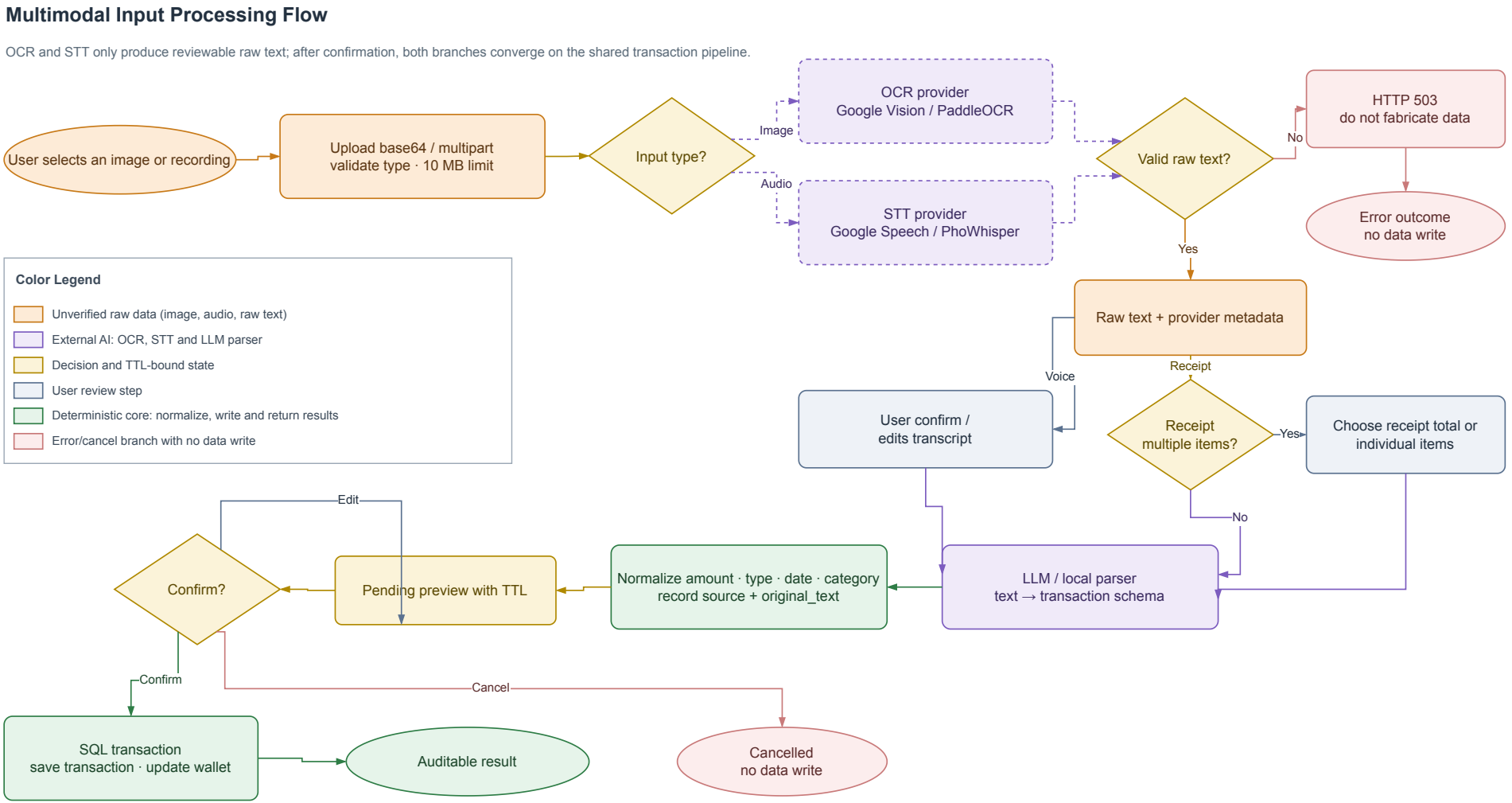

Figure 8: Image and speech processing; both branches converge on the text pipeline after raw-text review.

OCR/STT returns only raw text and provider metadata. Current ground truth is too small and receipt line totals are not automatically reconciled, so Figure 8 is a design flow, not evidence of accuracy.

3.2.3.2. Classification and Correction Retrieval

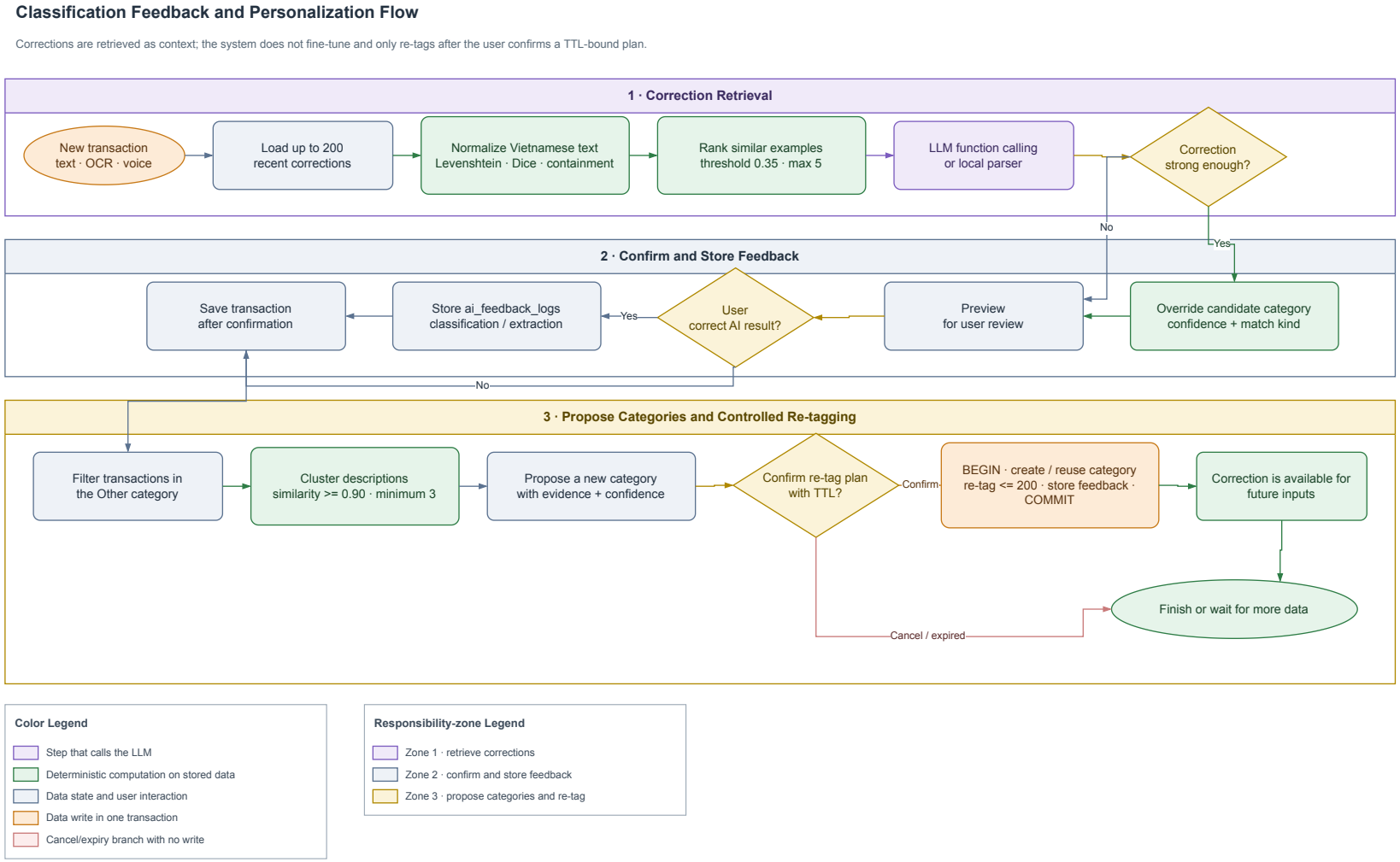

Figure 9: Classification and correction-retrieval flow; feedback is stored only after confirmation.

The matcher uses

$$s = \max(s_{edit},\ 0.92 s_{dice},\ s_{contain}). \tag{3.1}$$

Let U, V be normalized token sets and choose $|U| \le |V|$. The implementation defines containment as

$$s_{contain} = \begin{cases} \min\left(0.94,\ 0.82 + 0.12\dfrac{|U|}{|V|}\right), & U \subseteq V \text{ and } |U| \ge 2, \\ 0, & \text{otherwise.} \end{cases} \tag{3.2}$$

Selection proceeds through exact name, exact alias and fuzzy matching. Input length is measured after normalization: input longer than four characters uses $\tau = 0.82$, shorter input uses $\tau = 0.90$, and the best candidate must exceed the second by $m = 0.08$. These are prototype starting values, not universal optima. Retrieval loads at most 200 recent corrections and overrides only with a record of the same income/expense type; legacy records without type are ineligible. A fuzzy correction needs at least 0.88 similarity, and competing labels require at least 0.8 agreement for the winner. Multiple labels for one exact text return fallback/clarification instead of majority voting.

3.2.3.3. Analytics and Planning Algorithms

Default windows are 14 calendar days for runway, 30 calendar days for anomaly, 200 days for recurring discovery, six completed months for trend, 60 calendar days for day-of-week analysis and 12 completed ISO weeks for correlation. Fixed day/month/week axes zero-fill inactive periods; an incomplete current month/week is excluded from comparisons. A payday is a local day-of-month and day 31 is clamped to the target month's last calendar day.

Runway uses only liquid VND balance:

$$B = \sum_{\substack{w.currency=\text{VND} \\ w.type \in \{\text{cash,bank,e\_wallet}\}}} balance_w. \tag{3.3}$$

For daily expense e_j and $W = 14$,

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{3.4}$$

If $B \le 0$, then $d = 0$; if $\bar{e}_W = 0$, the depletion date is undefined. For monthly spending y_j, the trend detector keeps the zero-filled six-month axis but requires at least three positive observed months, then uses OLS and emits a signal only when slope

$b > 0$, $R^2 \geq 0.5$, and mean stepwise percentage change (over steps with a positive preceding value) is at least 10%.

Anomaly detection uses the sample standard deviation over all 30 days. It is evaluated only when at least four calendar days have positive spending, and flags only the upper tail when

$$z_i = \frac{e_i - \bar{e}}{s} \geq 2.5 \quad \text{or} \quad e_i > Q_3 + 1.5\,IQR. \tag{3.5}$$

When $s = 0$, the implementation sets $z_i = 0$; when $IQR = 0$, the IQR branch is disabled. Thus a constant series cannot become anomalous merely because the denominator is undefined.

Recurring grouping normalizes descriptions and simultaneously requires at least three occurrences, every $\delta_i = |a_i - \bar{a}|/\bar{a} \leq 0.15$, and gap coefficient of variation $CV_g = \sigma_g/\bar{g} \leq 0.12$ (population σ_g over positive gaps). Every gap must fit one cadence: weekly 5–9 days, monthly 26–35 days or quarterly 80–100 days. A candidate is active only when its latest occurrence is no older than that cadence's maximum gap; the output also records `lastSeen` and the calendar-clamped `nextExpected`. There is no default amount ceiling. The monthly estimate is

$$\widehat{a}_{month} = \begin{cases} \bar{a}\,52/12, & \text{weekly}, \\ \bar{a}, & \text{monthly}, \\ \bar{a}/3, & \text{quarterly}. \end{cases} \tag{3.6}$$

Correlation zero-fills a shared 12-week axis, keeps categories observed in at least four weeks and skips undefined Pearson pairs (fewer than three paired observations or zero variance). It returns only the strongest positive pair when $r \geq 0.6$; association does not imply causation. Day-of-week analysis uses the 60-day window, requires at least four observed weekdays and emits a signal only when the peak-to-mean ratio reaches 1.8.

For category spending $s_{c,j}$ across m months, a five-percent-buffer baseline is

$$b_c = 1.05 \left(\frac{1}{m} \sum_{j=1}^{m} s_{c,j} \right). \tag{3.7}$$

Month-end forecast is $\widehat{S}_D = (S/d_e)D$, where S is spending so far, d_e elapsed calendar days including today and D month length. The 50/30/20 and hybrid strategies shrink raw allocations when they exceed a cap; post-rounding reconciliation keeps each group at or below its cap. Needs, wants and savings rates may not sum to more than 1; recommendations require confirmation.

For a saving or purchase goal with remaining amount R and monthly contribution $M > 0$, $n_s = \lceil R/M \rceil$. For current debt principal L and nominal annual percentage rate i, the implementation uses monthly rate $r = i/(12 \times 100)$. With a debt deadline of n_d months, the required end-of-month level payment is

$$P = \frac{Lr}{1 - (1 + r)^{-n_d}} \tag{3.8}$$

for $r > 0$, and $P = L/n_d$ for $r = 0$. A payment less than or equal to first-month interest triggers a negative-amortization warning; a zero payment is reported separately. Without a deadline, the implementation simulates month by month up to a 600-month default horizon. What-if currently reruns the saving/purchase function with an additional contribution; debt plans are not changed by that scenario, and no what-if result is persisted.

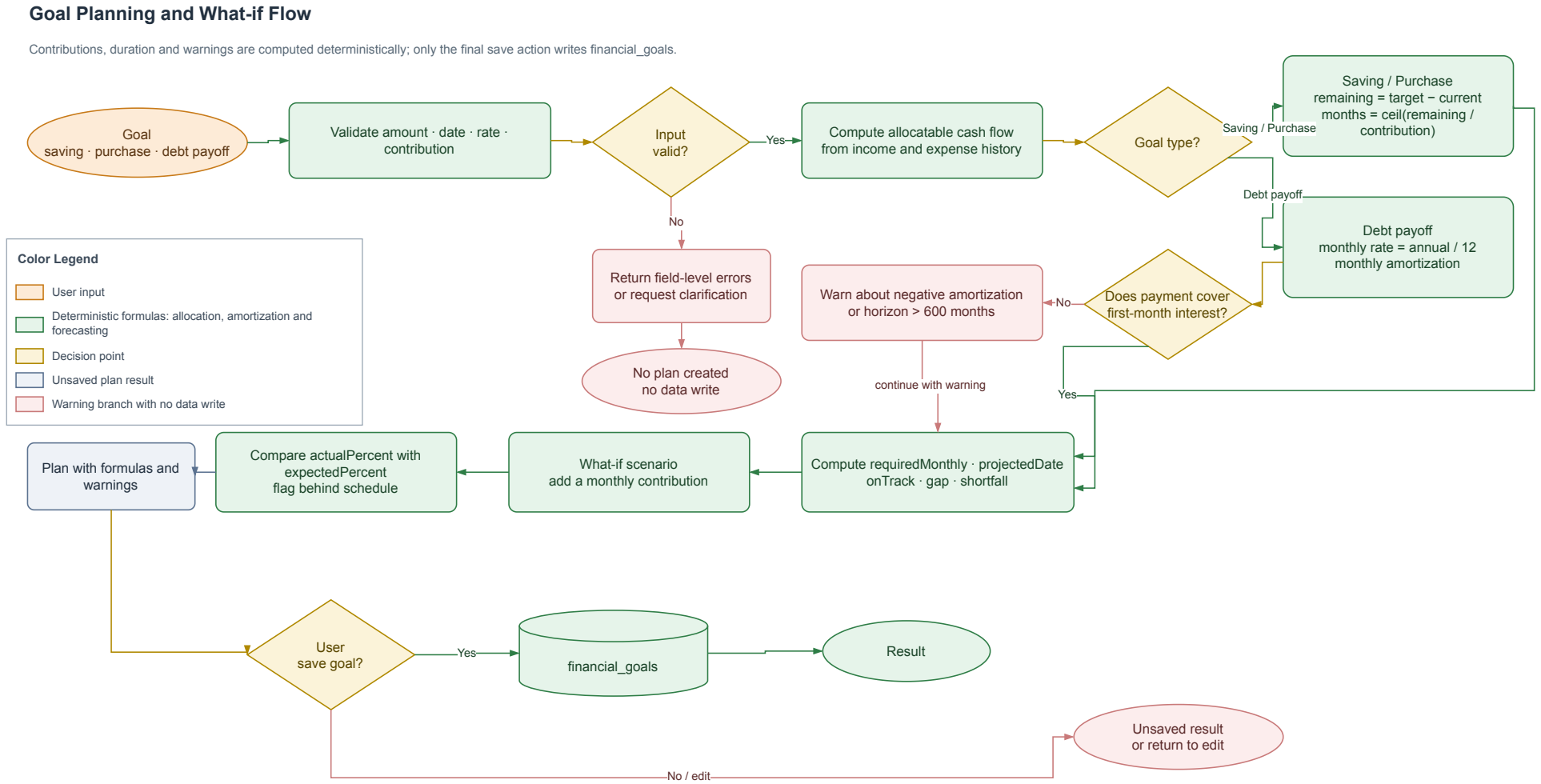

Figure 10: Goal planning and what-if flow; only final confirmation persists a goal.

3.2.3.4. Grounded Insight Generation

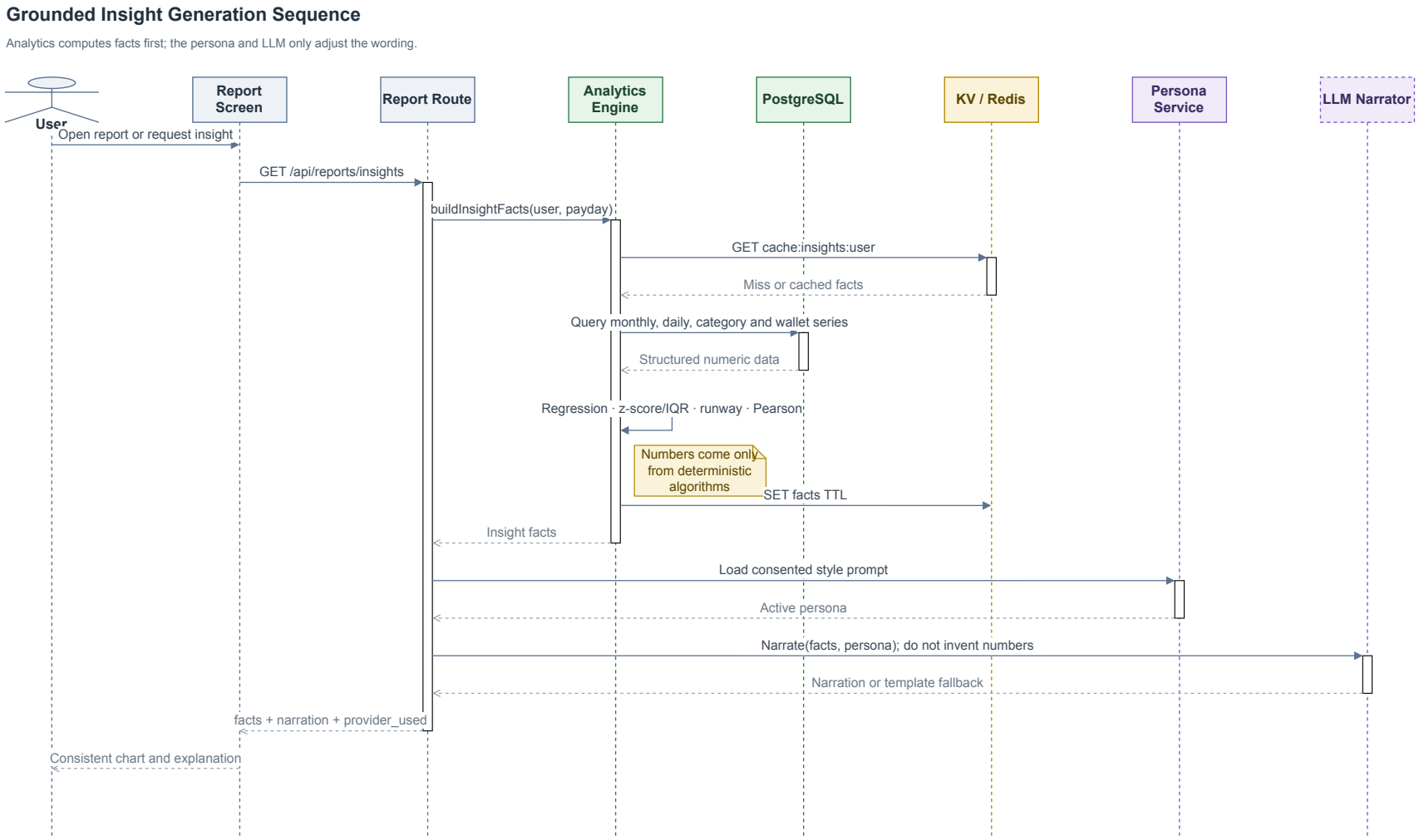

Figure 11: Grounded-insight sequence; Analytics returns facts before persona/LLM narration.

Facts contract version 1.0 returns `schema_version`, `generated_at`, each component value and `metadata.components`. Per-component metadata contains `unit`, `window`, `sample_count`, `method`, `parameters`, `warnings`, and `status` in `ok/no_signal/degraded`. The window object records `size`, `unit` and `zero_filled`; selected components add `category_count` or `wallet_count`. A local failure appears in `degraded_components` without removing the contract. A persona changes tone only with consent; the response includes narration and facts for UI/test comparison. A numeric checker covering every expression remains a target design.

The calculation path, numeric examples and boundary cases for matching, correction, Analytics, Budget, Goal and the facts contract are expanded in Appendix C–C.6; this chapter keeps the main formulas and design decisions for a coherent core narrative.

3.3. TESTING

3.3.1. Test Plan

Table 25: Test-plan matrix

Level	Target	Technique	Expected evidence
Unit	Matching, analytics, budget, goal and validation.	Normal, boundary, hand-computed expected values and invariants.	`node:test` pass/fail by case.
Integration	Route–service–model, PostgreSQL and Redis.	Test DB, rollback, TTL and concurrent claim.	Logs, DB snapshots and exit code.
System	Text/voice/image through preview and DB.	End-to-end scripted smoke.	HTTP response, DB assertion and UI image.
AI contract	Parser, tool schema, LLM and corrections.	Joint type/category labels; abstentions stay in the denominator; whole-group split.	JSON records, coverage, accuracy/F1 and helped/harmed/net.
Media	OCR/STT adapters.	Ground-truth image/audio.	CER/WER and field accuracy.
Worker	Retry, deduplication and schedule.	Repeated fingerprint and fault injection.	Before/after effect count.

Level	Target	Technique	Expected evidence
UAT	Target users.	Timed tasks and step count.	Completion, duration and SUS.

The text dataset must cover currency units, written numbers, multiplication, relative dates, multiple transactions, typos, mixed Vietnamese/English, missing or ambiguous fields and conflicting corrections. In correction evaluation, every row sharing transaction type and normalized description must lie wholly in seed or evaluation. Analytics data needs known slopes, outliers, cadence and correlation. Media data must store image/audio with a reference transcript or receipt table.

The normal-case, boundary-case, invariant and test-file catalog is provided in Appendix C.7; the source, schema, cohorts and dataset limitations are documented in Appendix C.1. These appendix tables point to evidence without replacing the measured results in Section 3.3.2.

3.3.2. Results and Analysis

3.3.2.1. Measured Operational Results

Table 26: Measured results and conclusion boundaries

Item	Result	Status	Permitted conclusion
Backend `npm test`	44/44 test files pass.	**Measured**	Node 24 reports file-level results; this is neither 182 test cases nor UAT/production evidence.
Local-parser gate	31/31 strict pass.	**Measured**	Fixed set of 31 sentences; not an independent benchmark.
Full API/DB/media smoke	23/23 checks pass.	**Measured**	Selected fixtures traverse HTTP–PostgreSQL/provider; Redis worker excluded.
Mobile-web UI smoke	4/4 gates pass at 390×844.	**Measured**	No overflow and images render; not UAT.

Item	Result	Status	Permitted conclusion
Data import	5,265 rows, 0 rejects.	**Measured**	Provenance-aware importer reconciled on demo PostgreSQL.
Parser on 5,265 rows	Joint type/category: accuracy 26.12%; macro-F1 0.1559.	**Measured**	Category-only 29.36%/0.177 is an auxiliary comparison.
Parser vs Gemini	Full-set accuracy 22.22% vs 39.68%.	**Measured**	Gemini coverage is 42/63; macro-F1 0.6068 is conditional on those 42 answers.
Correction retrieval	Conditional on parser type correctness and excluding *Other*: accuracy 34.46% $\rightarrow$ 61.22%; coverage 891/2,612; net $+26.76$ points.	**Measured**	4,756 eligible rows; 76 wrong-type rows excluded; whole-group type+normalized-text split; zero seed/evaluation overlap.
OCR/STT accuracy	Not reported.	**Not measured**	Two images and one audio smoke fixture cannot establish CER/WER/field accuracy.
Live Redis worker	Not confirmed.	**Not measured**	Payment logic is tested, but live queue/worker was not measured.
Grounding, p95 and UAT	Not reported.	**Not measured**	Requires checker, at least 30 runs per flow and target users.

Smoke testing only establishes completion of selected scenarios. It is not OCR/STT accuracy evidence, and a tested payment handler does not prove that a live Redis worker ran.

3.3.2.2. Classification Experiments

The experiments use the 5,265-row `dataFinance.csv`. Classification and correction were rerun offline on 2 August 2026; the Gemini ablation is a historical measurement from 24 July with a 2 August reanalysis artifact. Code and JSON/Markdown artifacts reside under `demo/backend/tests/experiments/` and `log/`.

Table 27: Local-parser benchmark on 5,265 transactions

Metric	Value
Joint type/category accuracy	26.12%
Joint macro-F1 (13 represented classes)	0.1559
Joint weighted-F1	0.2913
Incorrect joint predictions	3,890/5,265
Errors involving Other	3,128
Wrong income/expense type	427
Correct category name but wrong type	171
Category-only accuracy (auxiliary)	29.36%
Category-only macro/weighted-F1 (auxiliary)	0.1770/0.3013

The primary label joins transaction type and category name, so `income/Other` and `expense/Other` are distinct. Most errors still involve Other because some historical labels describe funding source while the parser classifies content. The gap from category-only metrics also shows why a correct name with the wrong income/expense direction cannot count as correct.

Table 28: Local parser and Gemini ablation on the same 63 sentences

Branch	Coverage	Full accuracy	Answered accuracy	Answered macro-F1	p50
Local parser	63/63	22.22%	22.22%	0.2039	0 ms
Gemini	42/63	39.68%	59.52%	0.6068	964 ms

The historical runner removed 21 `null` categories before computing 59.52% and macro-F1 0.6068, so both describe only the 42 answered samples. Counting abstentions as incorrect, Gemini gets 25/63 (39.68%) on the full set. The old artifact lacks every sample-level prediction, so full-set macro-F1 cannot be reconstructed and remains unavailable. All 63 calls completed without a recorded provider error, but 21 outputs had no category. This small sample and 964-ms p50 support further LLM experiments, not universal superiority.

3.3.2.3. Correction-retrieval Experiment

After removing *Other*, 4,832 samples are partitioned into whole normalized-text groups within each transaction type. Category correction is evaluated only when the parser transaction type equals the gold type: 4,756 rows are eligible and 76 wrong-type rows are excluded because category retrieval cannot repair the type. Group keys use `normalizeForMatch`. The selected seed side contains 1,714 parser-wrong records and 430 parser-correct co-members (2,144 rows across 636 groups); the greedy group selection targeted 1,713 parser-wrong records but cannot split a group. Evaluation retains 1,712 parser-wrong holdout records and 900 parser-correct controls outside all seed groups; those 430 co-members are excluded even though they do not become corrections. Seed/evaluation key overlap is zero, eliminating exact-match leakage.

Table 29: Correction retrieval on the 2,612-sample group-split evaluation

Metric	Before	After	Difference
Accuracy	34.46%	61.22%	+26.76 points
Macro-F1	0.3080	0.5898	+0.2818
Weighted-F1	0.3990	0.7072	+0.3082
Correction coverage	0%	34.11%	891/2,612
Helped/harmed/net	–	740/41/699	net +26.76 points

All 891 applications are fuzzy; exact applications are zero as required by the split. In the parser-wrong holdout cohort, accuracy rises from 0% to 43.22% (740 helped cases among 1,712; 796 corrections applied). Among 900 parser-correct controls, 95 corrections apply and 41 cause errors, leaving 95.44% accuracy. One of the 41 flips is consistent with conflicting historical labels and 40 are genuine harmful flips. The net 699 cases over 2,612 equal the +26.76-point accuracy change. This controls leakage and measures both benefit and harm, but still uses one historical dataset, one seed and no confidence interval; it is not a production estimate.

3.3.3. Requirements-to-Evidence Traceability

Table 30 links observable requirements to design, tests and evidence status. A route, mock or diagram alone is not end-to-end acceptance evidence.

Table 30: FR–design–test–evidence traceability

Req.	Design	Related test	Evidence and gap
FR-01, FR-03–FR-06	Typed draft, TTL preview, service and SQL transaction.	Parser gate, backend tests and API–DB smoke.	Selected fixtures and rollback invariants pass; no independent task-based user measurement.
FR-02	Media adapter, raw-text review and shared FR-01 pipeline.	Upload/media smoke; planned CER/WER and field accuracy.	Sample provider flow executes; insufficient ground truth for OCR/STT accuracy.
FR-04	Joint classifier, type-scoped corrections, conflict thresholds and group split.	Classification benchmark, ablation coverage and correction helped/harmed/net.	Record-level artifacts and zero overlap exist; historical labels, small sample and no confidence interval still limit conclusions.
FR-07–FR-10	Analytics, facts contract, Budget/Goal service and grounded narrator.	Hand-expected unit tests, metadata contract and planned checker.	Formulas, calendar windows and metadata have tests; response-wide grounding and representative threshold calibration remain incomplete.
FR-11	Recurring service, BullMQ, event key and retry.	Payment/handler unit tests; planned live worker and fault injection.	Business logic is tested; no live queue/worker evidence in this evaluation.
FR-12	CSV exporter, history and TTL cleanup.	Escaping tests, API smoke and planned expiry check.	CSV has evidence; the former PDF-labelled path only generates HTML.

Req.	Design	Related test	Evidence and gap
NFR-01–NFR-08	Constraints, user scope, fallback, idempotency and fact metadata.	Rollback, cross-scope, load, recovery, worker and UAT.	Test integrity has evidence; three-hour recovery, p95, UAT, live worker and faithfulness remain targets.

3.3.4. Threats to Validity

3.3.4.1. Internal Validity

Benchmarks depend on historical labels, and identical descriptions may have different categories. The new correction experiment splits whole type+normalized-description groups with zero overlap, but seed and evaluation still come from one dataset, use one seed and have no confidence interval. The 63-sentence ablation is small; its historical artifact omitted complete prediction records and computed conditional metrics after removing 21 abstentions, so full-set Gemini macro-F1 cannot be recovered. Providers also vary by version and time; the updated runner therefore stores model, parameters, timestamp and sanitized structured records.

3.3.4.2. Construct Validity

The 31/31 gate measures success on designed sentences, not out-of-sample correctness. Smoke tests measure scenario completion, not accuracy or usability. Accuracy can hide rare classes, while abstention can inflate conditional metrics. Coverage, macro-F1, weighted-F1, per-class confusion, helped/harmed/net and support are therefore reported together. For OCR/STT, CER/WER alone is insufficient: amount, date and merchant field accuracy maps more directly to ledger risk.

3.3.4.3. External Validity

Data comes from one demo profile, uses VND and is primarily Vietnamese. Results do not automatically generalize to many users, other taxonomies, varied receipts or production load. The next evaluation should lock the taxonomy, sample randomly over time, retain group splitting while running multiple seeds/confidence intervals, and add target-user UAT. Conclusions apply only to the source, dataset and environment recorded in the 24 July ablation and 2 August 2026 classification/correction artifacts.

CHAPTER 4: CONCLUSION

4.1. OBJECTIVE COMPLETION

PERFIN now forms a prototype with a clear separation between data, algorithms and the LLM layer. It contains 18 business tables plus the technical `_migrations` table, transaction, analytics, feedback, budget, goal and state modules, and an automated test suite. Its central contribution is not replacing business logic with an LLM, but using the model as a language-conversion layer while the backend retains validation, computation and data-write authority.

Table 31: Objectives and evidence

Code	Main result	Conclusion
O1	Migrations, ERD, constraints and tested transactional flows.	Prototype-complete; broader DB fault injection is needed.
O2	Text/media adapters, typed draft, clarification, preview and confirmation.	Flow executes; OCR/STT accuracy is unmeasured.
O3	Trend, anomaly, runway, recurring, correlation, budget and goal planner.	Implemented with unit tests; thresholds need representative calibration.
O4	Tool schema, service validation, facts–narrator flow and fallback.	Boundary implemented; complete numeric faithfulness is unmeasured.
O5	44/44 backend test files, 23/23 full smoke and experiments with coverage/group splitting.	Evidence supports the demo core, not live worker, p95, UAT or production.

On 5,265 transactions, the parser obtains 26.12% accuracy and macro-F1 0.1559 under joint type/category labels; category-only 29.36%/0.177 is auxiliary. In the 63-sentence ablation, full-set accuracy is 22.22% for the parser and 39.68% for Gemini, but Gemini returns a category for only 42/63 sentences; macro-F1 0.6068 is conditional on those 42. Conditional on a correct parser transaction type and excluding *Other*, correction group splitting has zero overlap and raises accuracy from 34.46% to 61.22% on 2,612 evaluation rows (4,756 eligible; 76 wrong-type rows excluded), with 740 helped, 41 harmed, net $+26.76$ points and 891/2,612 coverage. This is evidence for the disclosed dataset/seed, not a production estimate.

4.2. DELIVERABLES

Deliverables include mobile/backend source, PostgreSQL migrations, Redis/BullMQ configuration, LLM/OCR/STT adapters, tests, a reconciled demo dataset, experiment artifacts and this LaTeX report. The prototype supports text/media entry with review, ledger management, analytics, budgets and goals, and returns facts with controlled narration.

On the small ablation sample, Gemini has higher full-set accuracy than the parser but abstains on one third of inputs and adds latency/provider dependence; full-sample macro-F1 cannot be recovered from the old artifact. This potential value is safe only behind four controls: schema, validation, preview/confirmation and fact grounding. PERFIN is therefore a financial-data system with an LLM interaction layer, not a chatbot that calculates or decides on the user's behalf.

4.3. LIMITATIONS

1. The prototype uses `default_user` and lacks production authentication and authorization.
2. Smoke tests on two images and one audio file cannot establish CER, WER or transaction-field accuracy.
3. Live Redis worker behaviour, retry/idempotency, p95 and UAT were not confirmed in a version-locked environment.
4. Fuzzy, anomaly, recurring and correlation thresholds and time windows are initial configurations rather than universal optima.
5. Historical taxonomy is inconsistent; the ablation lacks full prediction records, while correction has only one seed and no confidence interval.
6. The numeric-grounding checker remains target design, and persona effects have not been evaluated.
7. Remaining gaps include true PDF export, full backup/restore, push notification and complete fresh-install wallet creation.

4.4. FUTURE WORK

Next priorities are to lock taxonomy, create an independent evaluation set and run group splits over multiple seeds/confidence intervals; build image/audio ground truth for CER/WER and field accuracy; run the Redis worker with fault injection and duplicate-effect checks; implement numeric grounding and measure p50/p95 over at least 30 runs per flow; conduct UAT comparing chat and forms; and add authentication, authorization, audit and secret management before real deployment.

Bank synchronization, shared wallets and high availability should follow only after data correctness and empirical evidence stabilize. This ordering preserves the basic-topic project's intended focus: data and algorithms are the core, while the LLM is a replaceable, measurable supporting component.

BIBLIOGRAPHY

[1] A. Lusardi and O. S. Mitchell, "The Economic Importance of Financial Literacy: Theory and Evidence," *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[2] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[3] L. R. Dice, "Measures of the Amount of Ecologic Association Between Species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[4] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed. Hoboken, NJ, USA: Wiley, 2012.

[5] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[6] K. Pearson, "Note on Regression and Inheritance in the Case of Two Parents," *Proceedings of the Royal Society of London*, vol. 58, pp. 240–242, 1895.

[7] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. 9th International Conference on Document Analysis and Recognition*, vol. 2, pp. 629–633, 2007.

[8] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[9] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] Gemini Team, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.

[11] Google AI for Developers, "Function calling with the Gemini API," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling. [Accessed: 15-Jul-2026].

[12] PostgreSQL Global Development Group, "PostgreSQL Documentation — Transactions and Data Definition," 2026. [Online]. Available: https://www.post

gresql.org/docs/current/tutorial-transactions.html and https://www.postgresql.org/docs/current/ddl.html. [Accessed: 15-Jul-2026].

[13] Redis Ltd., "EXPIRE," *Redis Commands Documentation*, 2026. [Online]. Available: https://redis.io/docs/latest/commands/expire/. [Accessed: 15-Jul-2026].

[14] Taskforce.sh, "BullMQ Documentation: Idempotent Jobs and Retrying Failing Jobs," 2026. [Online]. Available: https://docs.bullmq.io/patterns/idempotent-jobs and https://docs.bullmq.io/guide/retrying-failing-jobs. [Accessed: 15-Jul-2026].

[15] Money Lover, "Money Lover — Money Manager, Budget Expense Tracker," 2026. [Online]. Available: https://moneylover.me. [Accessed: 24-Jul-2026].

[16] MISA JSC, "MISA MoneyKeeper — Personal Expense Management Application," 2026. [Online]. Available: https://www.misa.vn/moneykeeper. [Accessed: 24-Jul-2026].

[17] ISO/IEC/IEEE, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE Std 29148:2018, 2nd ed., 2018.

CHAPTER A: INSTALLATION GUIDE

The minimum environment is Node.js/npm, PostgreSQL and Expo Go or a browser. Redis 7 is recommended for the worker. Without Redis, the API may use an in-memory KV fallback during development, but queue and retry behaviour is not demonstrated.

A.1. BACKEND SETUP

From the project root, copy the sample configuration, supply PostgreSQL settings and retain `AI_PROVIDER=local` when no Gemini key is used. Never commit secrets or service-account files.

```
cd demo/backend
cp .env.example .env
npm install
npm run migrate
npm start
```

The API defaults to http://127.0.0.1:3000. Use that URL for a basic availability check. Recreate schema and seed data only on a development/test database:

```
npm run migrate:fresh
```

A.2. REDIS AND WORKER

From `demo`, start Redis with Docker Compose and open another terminal for the worker:

```
docker compose up -d redis
cd backend
npm run worker
```

Verify `REDIS_URL`, `JOBS_ENABLED` and timezone in `backend/.env`. `docker compose down` stops Redis while retaining its volume.

A.3. FRONTEND SETUP

```
cd demo/frontend
npm install
EXPO_PUBLIC_API_URL=http://127.0.0.1:3000 npm start
```

For a phone on the same LAN, replace `127.0.0.1` with the backend host's LAN address and use `npm run start:lan`. If the device cannot reach that address, use Expo tunnel plus a separate backend tunnel. Never expose AI secrets through frontend environment variables.

CHAPTER B: QUICK USER GUIDE

1. Open the application and check that Dashboard shows balance, total income/expense and recent transactions.
2. Add a transaction from Transactions, or open Chat and enter a sentence such as "ăn phở 50k sáng nay".
3. For image/audio, review and edit the OCR text or transcript. Media results are not stored automatically.
4. Review amount, type, date, category and wallet in the preview; edit, cancel or confirm. Only confirmation creates a transaction.
5. Use Budget for limits, Reports for trends/insights and Goals for plans and what-if scenarios.
6. Correct an incorrect category. Confirmed corrections may be retrieved for a later input.

The prototype has no login and uses `default_user`. Do not enter sensitive data or treat results as guaranteed investment advice.

CHAPTER C: EXTENDED ALGORITHM SPECIFICATIONS

This appendix expands the formulas and rules summarized in Chapter 3. It lets a reader follow the path from scoped input data to facts or a plan; it does not introduce a second source of truth or change prototype behavior. Examples use synthetic values, while names, thresholds and statuses are checked against the current implementation.

C.1. TEST DATASET DESCRIPTION

This section documents the data sources used for tests and benchmark measurements. The main data is not an external, independently collected test set: classification and correction experiments both read `demo/data/dataFinance.csv`, then apply different cohorts or splits. The report publishes aggregate statistics and checksums only; raw transaction descriptions are not copied into the appendix.

C.1.1. Dataset inventory

Table 32: Test data sources and roles

Source	Size/scope	Role and status
Transaction CSV	5,265 rows, 8 columns; 1 Jan 2022–15 Jul 2026; 4,845 expenses and 420 incomes.	`dataFinance.csv` is the main source for import, parser benchmarking and correction input. The importer accepted 5,265/5,265 rows.
Category mapping	Crosswalk from the historical taxonomy to PERFIN's 17 type/category labels.	`dataFinance.category-map.json` is checked against the CSV checksum and used to create post-import gold labels, not parser predictions. Only 13/17 labels have support in the benchmark.
Parser–Gemini ablation	At most five sentences per class, seed 42; 63 sentences in the measured run.	A stratified cohort sampled from the same CSV to measure coverage, accuracy and F1; it is not an independent text source.

Source	Size/scope	Role and status
Correction group split	4,832 rows after removing *Other*; 4,756 same-type eligible rows; 2,612 evaluation rows.	Seed/evaluation are split by whole transaction-type and normalized-description groups, seed 7, target ratio 0.5; key overlap is zero.
Media fixtures	Two images and one audio file. The images have expected amount/type/date values; the audio has no reference transcript.	`media-fixtures.json` is used only for media smoke. Without ground truth, the result is pipeline pass/fail, not OCR/STT accuracy.
Analytics/unit fixtures	Synthetic or anonymized series with known slope, outlier, cadence and correlation.	Used to check Analytics/Budget/Goal formulas, edge cases and invariants; not a real-user benchmark.

C.1.2. Raw schema and category crosswalk

The CSV must contain the following eight columns in this exact order. The `planFinanceCsvImport` function validates the schema, dates, amounts, direction and mapping checksum before creating normalized records.

Table 33: CSV schema and post-import fields

CSV column	Valid values	Transformation and meaning
`Title`	Non-empty string, at most 200 characters.	Free-text transaction description; whitespace is normalized into `description` while the original is retained in `original_text`.
`Budget`	Non-zero number; positive for income and negative for expense.	Absolute value becomes `amount`; it must equal the absolute value of `Cost`.

CSV column	Valid values	Transformation and meaning
`Cost`	Positive VND string, optionally containing ₫.	Parsed as a positive integer and cross-checked against `Budget`; it is not a second money measure.
`Date`	Existing date in `DD/MM/YYYY` format.	Converted to ISO `transaction_date` in `YYYY-MM-DD` form.
`Ex/In`	`Expenses` or `In-come`.	Maps to `type=expense` or `type=income`; other values are rejected.
`Special`	Optional string; may be blank.	Stored as `special` metadata; a blank cell becomes `null`.
`Type Expenses`	Historical expense taxonomy; populated only when `Ex/In` is `Expenses`.	Looked up in the crosswalk to create `categoryName`; one expense row without a source label is intentionally mapped to *Other*.
`Type In come`	Historical income taxonomy; populated only when `Ex/In` is `In-come`.	Looked up in the crosswalk to create `categoryName`; the opposite-direction category column must be blank.

The importer does not silently remove duplicates: the source contains 23 exact-duplicate groups, 24 extra rows in total, but has no ID or timestamp sufficient to call them errors, so the default mode retains them. The mapping declares the CSV checksum `a9b7cf1b...`; the benchmark artifact stores the full checksum, and changing one CSV byte stops the import instead of applying an old mapping to new data.

C.1.3. Size, distribution and data quality

Table 34: Importer quality summary for the transaction set

Attribute	Value	Interpretation
Source rows / imported rows	5,265 / 5,265	No row was rejected in the reconciled import.
Transaction direction	4,845 expense; 420 income	The data is strongly expense-heavy; joint metrics retain direction in the label.
Date range	1 Jan 2022–15 Jul 2026	Year counts are 540 (2022), 1,567 (2023), 1,525 (2024), 1,050 (2025) and 583 (2026).
Normalized descriptions	374/5,265 rows	Unicode/whitespace normalization was applied before record creation and reused for group splitting.
Blank `Special` cells	4,978/5,265 rows	Missing optional metadata, not a validation error.
Missing source label	One expense row	Explicitly mapped to *Other*; opposite-direction category cells are blank by schema.
Amounts	min 1,000; median 25,000; max 48,000,000 VND	$Q1 = 14,000$, $Q3 = 44,000$, IQR threshold 89,000; 621 rows are flagged above the threshold but retained.
Exact duplicates	23 groups / 24 extra rows	Retained by default to preserve the source; no duplicate error is asserted without a transaction identifier.

After mapping, *expense/Ăn uống* is the largest class with 2,251/5,265 rows (42.75%). Four valid labels have no support: *expense/Tạp hóa*, *expense/Điện tử*, *expense/Làm đẹp* and *income/Thưởng*. The parser macro-F1 is therefore reported over 13 supported classes, not interpreted as coverage of all 17 taxonomy labels.

Table 35: Distribution of the 13 supported joint labels in the parser benchmark

Joint label	Rows	Share
expense/Ăn uống	2,251	42.75%
expense/Giải trí	660	12.54%
expense/Di chuyển	491	9.33%
expense/Thể thao	394	7.48%
expense/Hóa đơn & Dịch vụ	356	6.76%
income/Khác	350	6.65%
expense/Giáo dục	342	6.50%
expense/Nhà cửa	150	2.85%
expense/Khác	83	1.58%
expense/Sức khỏe	80	1.52%
income/Lương	67	1.27%
expense/Mua sắm	38	0.72%
income/Đầu tư	3	0.06%

C.1.4. Cohorts, splits and reproducibility

Table 36: Cohorts used by the reported measurements

Measurement	Denominator	Rule and cross-check artifact
Classification	5,265/5,265 rows	Run the local parser over the full CSV with no provider calls; the primary label is joint type/category. Artifact `log/classification-benchmark_2026-08-02.json`.
Parser–Gemini ablation	63 sentences	Stratified sample of at most five per class with seed 42; Gemini abstentions remain in the denominator. Historical artifact `log/ablation-parser-vs-llm_2026-07-24.json`.

Measurement	Denominator	Rule and cross-check artifact
Correction retrieval	2,612 evaluation rows	From 4,756 same-type, non-*Other* rows: seed has 1,714 parser-wrong rows plus 430 co-members; holdout has 1,712 parser-wrong rows and control has 900 parser-correct rows. Seed 7, group split, overlap 0. Artifact `log/feedback-before-after_2026-08-02.json`.
Analytics/ Budget/Goal unit	Synthetic cases	There is no single transaction benchmark for these components; fixtures set known slope, zero-fill, outlier, cadence, correlation, budget caps and what-if outcomes. Tests are indexed in Table 40.
Media smoke	Two images + one audio	The images have expected amount/type/date values (one also has category); the audio has no ground truth. Used only to verify pipeline execution.

The runners store data and mapping checksums, Node version, run time and involved source files under `log/`. The CSV represents one demo profile, is primarily Vietnamese and uses VND; raw content is not treated as anonymized material for public release. Both image fixtures are marked as containing PII and unsafe for distribution, while the audio transcript has not been reviewed. Because there is no external holdout, no confidence interval and insufficient media ground truth, the measurements describe the recorded dataset/artifacts only; they do not establish generalization or production OCR/STT accuracy.

C.2. SCOPE AND DATA CONVENTIONS

The pure functions in Analytics, Budget, Goal and Feedback do not access the database or the LLM directly. Services scope the data to a profile, convert it to flat arrays or objects, call deterministic helpers, and attach metadata. The following conventions are shared:

Table 37: Shared input conventions for the algorithms

Convention	Application
Data scope	Every query receives `userId/userKey`; records outside the profile do not enter a calculation.
Currency	Runway uses VND and sums only `cash`, `bank` and `e_wallet`; it never converts foreign currency.
Ledger	Soft-deleted transactions are excluded from reports; a negative balance remains valid.
Calendar axis	Days/months/weeks are generated for the complete fixed window; inactive periods are zero-filled, while an incomplete current month or week is excluded from comparisons.
Missing data	A function returns `null`, an empty list or `no_signal`; it does not replace missing evidence with an invented number.
Rounding	Plan amounts are rounded at output; group-cap invariants are checked again after rounding.
Writes	Preview, correction, recommendation and what-if are calculated results; persistence still requires the domain service and confirmation.

Default windows are 14 days for runway, 30 days for anomaly, 60 days for day-of-week analysis, 200 days for recurring discovery, six completed months for trend and 12 completed ISO weeks for correlation. The facts contract uses version `1.0`; a failure in one component does not discard the others.

C.3. CLASSIFICATION AND CORRECTION RETRIEVAL

C.3.1. Normalization and category scoring

`normalizeForMatch` applies Unicode NFD, removes diacritics, maps `đ/Đ` to `d/D`, lowercases, replaces non-alphanumeric runs with spaces, and collapses repeated spaces. Thus " Điện-thoại! " becomes `dien thoai`. Exact, alias and fuzzy comparisons all use the normalized string and consider only the same `type` when the caller provides a transaction type.

For two normalized strings, the matcher computes three scores:

1. **Edit score:** $s_{edit} = 1 - d_{lev}(u, v)/\max(|u|, |v|)$, where d_{lev} is Levenshtein distance.
2. **Dice score:** twice the number of overlapping tokens divided by the number of distinct tokens in the two token sets, then multiplied by 0.92.

3. **Containment score:** when the smaller token set is contained in the larger and has at least two tokens, use $\min(0.94, 0.82 + 0.12|U|/|V|)$; otherwise use 0.

The final score is

$$s = \max(s_{edit},\ 0.92 s_{dice},\ s_{contain}). \tag{C.1}$$

The decision pipeline is:

1. Scope categories by transaction type and select the "Khác" fallback (or the first category if that fallback is absent).
2. If a normalized category name equals the input, return `exact` with confidence 1.
3. If exactly one alias equals the input, return `alias_exact` with confidence 0.98; aliases resolving to multiple categories return the fallback with reason `ambiguous_alias`.
4. Rank fuzzy scores over each category name and its aliases. Inputs longer than four characters require $s \geq 0.82$; inputs of at most four characters require $s \geq 0.90$. If a runner-up exists, the score gap must be at least 0.08.
5. If the threshold or margin is not met, return the fallback with reason `below_threshold` or `ambiguous_fuzzy`; do not silently choose the nearest candidate.

For "ăn uống", normalization gives `an uog`; relative to `an uong`, Levenshtein distance is 1 over length 7, so $s_{edit} = 6/7 \approx 0.857$. The score exceeds the 0.82 threshold and, without a competing candidate, returns "Ăn uống" with `matchKind=fuzzy`. In contrast, the alias "điện thoại" points to both "Điện tử" and "Hóa đơn & Dịch vụ" and must return "Khác" for clarification.

Phrase alias lookup also requires word boundaries. When several aliases match, the alias with more tokens wins; a tie in token count and length across different categories returns `ambiguous_alias`. This prevents a short alias such as "an" from matching inside "quần áo".

C.3.2. Correction ranking

A correction is useful only when a log has `feedback_type=classification`, original text and a corrected category different from the original category. The label key prioritizes `type + normalized category name`; an id is used only when the name is absent. The service loads at most 200 recent candidates from the current profile.

For every candidate, the service computes `textSimilarity`, keeps an exact match with score 1 or a fuzzy match with a minimum score of 0.88, and removes

a different income/expense type. Candidates are grouped by corrected label:

$$w_k = \sum_{j \in k} s_j, \qquad agreement = \frac{\max_k w_k}{\sum_k w_k}. \tag{C.2}$$

The winning candidate exposes `support` (group count), `bestScore` (maximum score) and confidence rounded from $s_{best} \times agreement$. Retrieval is rejected when:

- the input is empty or no candidate reaches the threshold;
- identical normalized text has multiple corrected labels;
- the non-exact winner is below 0.88;
- a competing label leaves agreement below 0.8;
- a legacy log lacks `type` while the caller requests a specific type.

Few-shot examples use a separate policy: at most five by default, an absolute limit of 20, and a minimum score of 0.35. Conflicting labels for the same typed normalized input are discarded, and duplicate labels keep only the nearest example. Correction therefore is not majority voting and cannot change transaction type.

Normalization, typo, ambiguous alias, exact/fuzzy correction, label conflict and name-only/id+name cases are covered by `feedback.test.js`. Coverage, helped/harmed/net and group-split metrics are covered by `experiment-metrics.test.js`.

C.4. FINANCIAL ANALYSIS ALGORITHMS

C.4.1. Calendar axes and runway

The helpers create an axis first, initialize totals to zero, and then add rows that fall within that axis:

- `completeDailyTotals` completes the day axis;
- `completeMonthlyByCategory` completes each category's month axis;
- `completeWeeklyByCategory` completes the completed ISO-week axis.

Correlation uses completed ISO weeks; the week containing today is excluded. This preserves the denominator when a category is inactive during a period.

Runway first filters wallets to `currency=VND` and types `cash`, `bank` and `e_wallet`:

$$B = \sum_{w \in \text{eligible wallets}} balance_w. \tag{C.3}$$

For a 14-day window and daily expense e_j (zero-spend days remain zero),

$$\bar{e}_W = \frac{1}{W}\sum_{j=1}^{W} e_j, \qquad d = \left\lfloor \frac{B}{\bar{e}_W} \right\rfloor. \tag{C.4}$$

If $B \le 0$, the result is $d = 0$ with `alreadyDepleted`; if $\bar{e}_W \le 0$, the result has `daysLeft=null` and no depletion date is invented. Payday is only a comparison warning for the next occurrence; day 31 is clamped to the last day of the target month.

For two liquid VND wallets totaling 300,000 VND and 14 days each spending 100,000 VND, $\bar{e}_W = 100,000$ and $d = 3$. USD and investment wallets do not increase B.

C.4.2. Trend and anomaly

For a zero-filled series $y_0, \ldots, y_{n-1}$, OLS uses $x_i = i$, $\bar{x}$ and $\bar{y}$ to calculate

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad a = \bar{y} - b\bar{x}. \tag{C.5}$$

It computes the fit statistic $R^2 = 1 - SS_{res}/SS_{tot}$. The next-step forecast is $\max(0, \text{round}(bn + a))$. Money slope/intercept are rounded and R^2 is rounded to three decimals. The engine emits a trend only with at least three positive-spend months, $b > 0$, $R^2 \ge 0.5$ and an average stepwise percentage change of at least 10% over steps with a positive preceding value. A constant series has $R^2 = 0$ and produces no signal.

Anomaly detection uses the mean, sample standard deviation and interpolated quantiles over the complete 30-day axis:

$$z_i = \frac{e_i - \bar{e}}{s}, \qquad upperFence = Q_3 + 1.5(Q_3 - Q_1). \tag{C.6}$$

Only the upper tail is flagged when $z_i \ge 2.5$ or $e_i > upperFence$, and the engine requires at least four positive-spend days before calling the helper. When $s = 0$, $z_i = 0$; when $IQR = 0$, the IQR branch is disabled. In the synthetic sample $[100, 100, 100, 1000]$, $Q_1 = 100$, $Q_3 = 325$ and the upper fence is 662.5, so 1000 is flagged by IQR even though its z-score is below 2.5.

C.4.3. Recurring and day-of-week analysis

Recurring discovery removes diacritics, lowercases, removes digits from descriptions and groups positive expense transactions by normalized description. A group needs at least three occurrences, every amount must be within 15% of the mean, and positive date gaps must have population coefficient of variation at most 0.12. All gaps must fit one cadence:

Table 38: Cadences used by recurring discovery

Cadence	Day gap	Monthly factor	Conversion example
Weekly	5–9	$52/12$	Four weekly payments are annualized over 52 weeks.
Monthly	26–35	1	The average payment is the monthly estimate.
Quarterly	80–100	$1/3$	A 900,000 VND quarterly payment becomes 300,000 VND/month.

A candidate older than its cadence maximum from `asOf` is inactive. The output keeps the latest description, occurrence count, average amount, cadence, `lastSeen`, `nextExpected` and `monthlyEstimate`; there is no default amount ceiling. Three 900,000 VND payments approximately one quarter apart therefore remain a quarterly recurring item estimated at 300,000 VND/month.

Day-of-week analysis uses spending per active day in the 60-day window; an inactive day is not treated as a zero-spend observation. It needs at least four observed weekdays and a peak-to-mean ratio of 1.8; otherwise it returns `null`.

C.4.4. Correlation

The engine creates a shared axis of 12 completed ISO weeks, zero-fills each category and keeps categories observed in at least four weeks. Pearson uses the shorter series length but returns `null` for fewer than three paired observations or zero variance:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}. \tag{C.7}$$

The engine scans every pair, keeps $r \geq 0.6$ and returns the pair with the greatest r. For proportional series $[100, 200, 300, 400]$ and $[200, 400, 600, 800]$, $r = 1$; the result describes co-movement in the selected window and does not imply causation.

C.5. BUDGET AND FINANCIAL-GOAL PLANNING

C.5.1. History summary and budget recommendation

`summarizeHistory` normalizes periods to `YYYY-MM`, aggregates income by month and aggregates expense by category id (or normalized name). With `expectedPeriods`, every listed month remains in the denominator even if it has no transaction. Thus a category spending 3,000,000 VND in one month of a three-month window has `average_spend=1,000,000`.

Each category is assigned to `needs` or `wants`; the default needs set contains food, transport, health, education, housing, bills/services and groceries, while `groupOverrides` can replace the assignment. With buffer q over m months,

$$b_c = (1+q)\frac{1}{m}\sum_{j=1}^{m} s_{c,j}. \tag{C.8}$$

The progress forecast receives `spent`, `amount_limit` and the target period. For the current month, $d_e = \max(1, \text{today's day})$; for a historical month, d_e is the full month length. If D is the number of days in the month,

$$\widehat{S}_D = \text{round}\left(\frac{S}{d_e}D\right), \qquad dailyRate = \text{round}\left(\frac{S}{d_e}\right). \tag{C.9}$$

The result records:

- `daily_rate`, `projected_spend` and projected percentage;
- the `likely_to_exceed` flag and, when applicable, `projected_exceed_day`.

When the limit is positive and current spending is below it, the service returns an exceedance day only if it remains in the month; a zero limit does not create a fictitious day. For 2,000,000 VND spent on day 10 of a 30-day month with a 5,000,000-VND limit, the rate is 200,000 VND/day, the forecast is 6,000,000 VND and the projected exceedance day is 25. The corresponding cases are in `budget-forecast.test.js`.

The strategies behave differently:

- **category average:** uses b_c directly.
- **50–30–20:** allocates group caps by each category's share of b_c, with default needs, wants and savings caps of $0.5I$, $0.3I$ and $0.2I$.
- **hybrid:** keeps b_c when the group total is within its cap; otherwise it scales all group items by $cap/groupAverage$.

Each raw limit is rounded by `roundingIncrement`, 10,000 VND by default. `enforceRoundedCap` then repeatedly reduces one increment from a positive category with the largest overshoot until the group total is within its cap. A sum

`needsRate+wantsRate+savingsRate` above 1 is rejected; missing income falls back to `category_average` with a warning.

For history with 10,000,000 VND monthly income, 4,000,000 VND needs and 4,000,000 VND wants over three months, with zero buffer, 50–30–20 gives caps of 5,000,000 and 3,000,000 VND. Hybrid keeps needs at 4,000,000 but shrinks wants to 3,000,000. If two wants categories are 15,000 VND each, a 10,000-VND rounding increment and a 30,000-VND cap still produce a post-reconciliation total within the cap.

C.5.2. Saving, debt payoff and what-if

For a saving/purchase goal, validation requires a positive target, non-negative current amount and a finite non-negative contribution. If contribution is `null`, the planner uses available surplus; an explicit user contribution of zero remains zero rather than being replaced by surplus. With remaining amount $R = \max(0, target - current)$ and monthly contribution $M > 0$,

$$n_s = \left\lceil \frac{R}{M} \right\rceil, \qquad projectedDate = addMonthsClamped(today, n_s). \tag{C.10}$$

End-of-month dates are clamped, so 31/01 plus one month becomes 28/02. If $R = 0$, the planner returns completed with horizon 0; if $M = 0$, horizon and projected date are `null` with `NO_MONTHLY_CONTRIBUTION`. A future deadline uses at least one contribution period in the same calendar month and reports `requiredMonthly`, gap and shortfall.

For current principal L, nominal annual percentage rate i and $r = i/(12 \times 100)$, the level end-of-month payment over n_d months is

$$P = \begin{cases} \dfrac{Lr}{1 - (1+r)^{-n_d}}, & r > 0, \\ \dfrac{L}{n_d}, & r = 0. \end{cases} \tag{C.11}$$

Without a deadline, the planner simulates each month: add interest, subtract a payment no larger than the amount due, and stop at zero balance or the horizon (600 months by default; validation permits at most 1200). A zero payment returns `NO_MONTHLY_PAYMENT`; a payment no larger than first-month interest returns `NEGATIVE_AMORTIZATION`; an unpaid balance after the horizon returns `MAX_HORIZON_EXCEEDED`. For $L = 10,000$, 12% annual interest and payment 1,000, the result is 11 months and 589.85 total interest; payment 100 is rejected because the balance does not decrease.

What-if accepts only saving or purchase plans, validates non-negative extra-

Monthly, and reruns the planner with original contribution plus the extra amount. It returns the new contribution, new horizon, months saved, feasibility transition and shortfall; it does not persist a goal or change a debt plan.

C.6. FACTS CONTRACT AND LLM BOUNDARY

The Analytics Engine runs six independent components: trend, anomaly, runway, subscriptions, day-of-week and correlation. Each reads scoped data and returns a value or `null`, then the engine wraps it in one contract:

Table 39: Fields in facts contract version 1.0

Field	Meaning
`schema_version`	Contract shape version, currently `1.0`.
`generated_at`	ISO timestamp at which facts were generated.
Component values	Numeric values or result lists; no signal is represented by `null`.
`metadata.components`	Each component includes `unit`, `window`, `sample_count`, `method`, `parameters`, `warnings` and `status`.
`status`	`ok` when a result exists; `no_signal` when the contract is valid but evidence is insufficient; `degraded` when the component failed.
`degraded_components`	Failed component names; no internal error message or credential is exposed.

The `window` object records size, unit and whether zero-fill was used; selected components add `category_count`, `wallet_count` or `observed_period_count`. Facts are cached for 600 seconds and reused only when schema, currency and payday context still match.

The numeric boundary is: SQL/Model reads source data, Analytics Engine computes facts, and only then does the narrator receive them. The LLM cannot call models, replace numbers, write transactions or turn `null` into a concrete value. A persona changes tone only when consent is enabled. A numeric grounding checker covering every numeric expression remains target design, not a measured result.

C.7. ALGORITHM–TEST MATRIX AND REPRODUCIBILITY

The following table points to tests and artifacts rather than copying raw data. “Checked” means that the selected test behavior passed; it is not production accuracy.

Table 40: Algorithm, boundary-case and evidence matrix

Group	Normal cases	Boundary cases/invariants	Evidence
Matcher	exact name, exact alias and clear typo.	Ambiguous alias, empty input, below threshold, small margin, aliases must use word boundaries.	`feedback.test.js`; historical local-parser gate 31/31.
Correction	exact/fuzzy same-type corrections and nearest few-shot examples.	Conflicting exact labels, missing type, agreement below 0.8, name-only/id+name.	`feedback.test.js`; correction JSON/Markdown in `log/`.
Calendar/runway	complete axis, liquid VND wallet and depletion date.	Zero balance, zero burn, USD/investment exclusion, day-31 payday.	`analytics-engine.test.js`, `analytics-time-series.test.js`, `wallet-policy.test.js`.
Trend/anomaly	increasing series and upper-tail outlier.	Fewer than 3 positive periods, fewer than 4 positive days, constant series, zero IQR or variance.	`analytics-engine.test.js`, `analytics-time-series.test.js`.
Recurring	cadence; strong pair.	Amount variation above 15%, gap CV above 0.12, stale candidate, fewer than 4 periods or undefined Pearson.	`subscription-miner.test.js`, `analytics-engine.test.js`.

Group	Normal cases	Boundary cases/invariants	Evidence
Budget	category average, hybrid and 50–30–20.	Inactive month remains in denominator, missing income, rate sum above 100%, rounded cap overflow.	`budget-recommender.test.js`, `budget-forecast.test.js`.
Goal	saving contribution, debt amortization and what-if.	Completed goal, zero contribution, end-of-month clamp, negative amortization, zero payment, horizon exceeded.	`goal-planner.test.js`, `goal-service.test.js`, `goal-validation.test.js`.
Facts contract	complete metadata and `ok` status.	Component failure preserves contract, `no_signal`, no error-message leak, mismatched cache context.	`analytics-engine.test.js`; contract version 1.0.

The classification and correction runners write structured records under `log/`; the 02/08/2026 measurements are described in Chapter 3.3.2. OCR/STT accuracy, complete numeric grounding, p95, UAT and live Redis-worker execution remain targets or unmeasured; they cannot be inferred from pure unit tests.

CHAPTER D: REPRODUCTION COMMANDS

Run each command from the named component directory. Record commit, Node version, provider configuration and PostgreSQL/Redis state without storing secrets in artifacts.

```
cd demo/backend
npm test
npm run test:ai
npm run smoke:full
node --test tests/experiment-metrics.test.js
node tests/experiments/classification-benchmark.js --out ../../log
node tests/experiments/feedback-before-after.js --out ../../log

cd ../frontend
npm run ui:smoke
npx expo export --platform web --output-dir /tmp/perfin-web
```

In the Node 24 artifact environment, `npm test` aggregates results by test file; 44/44 does not mean 44 or 182 individual test cases. The final test summary is stored at `log/backend-test-run_2026-08-02.json`. The final two runners are offline. Do not rerun the Gemini ablation without deliberately supplying provider access/quota; its historical measurement and reanalysis are under `log/`. Media smoke tests require a real provider and fixtures. Without ground truth, report availability pass/fail only. Integration tests need a separate test database and cleanup after each group.

CHAPTER E: DIAGRAM SOURCES

The report keeps 14 editable Draw.io sources and one overall use case. Every diagram label is English so the Vietnamese and English reports share the same set. Sources are in `latex/figures/drawio/`; PDF/PNG/SVG exports are in `latex/figures/rendered/`.

No.	Subject	Basename
1	System context	`01-system-context`
2	Runtime architecture	`02-runtime-architecture`
3	Prototype deployment	`03-deployment`
4	Domain class diagram	`04-domain-class`
5	Physical ERD	`05-physical-erd`
6	LLM boundary	`06-llm-boundary`
7	Conversation state	`07-conversation-state`
8	Text transaction sequence	`08-text-sequence`
9	Multimodal input	`09-multimodal-flow`
10	Feedback and classification	`10-feedback-flow`
11	Grounded insight	`11-insight-sequence`
12	Goal planner and what-if	`12-goal-flow`
13	Background job	`13-worker-sequence`
14	Overall use case	`14-usecase-overview`